



Advancements in automated program repair: a comprehensive review

Sena Dikici¹ · Turgay Tugay Bilgin¹

Received: 29 May 2024 / Revised: 12 January 2025 / Accepted: 18 February 2025 /
Published online: 6 March 2025
© The Author(s) 2025

Abstract

This review paper presents a comprehensive examination of automated program repair (APR) and its significant contribution to the field of modern software engineering. It elucidates how APR methodologies markedly mitigate manual debugging needs by automating the detection and resolution of software glitches. The study encompasses an in-depth exploration of three primary categories of APR techniques: template-based, machine learning, and deep learning approaches, drawing from an exhaustive evaluation of 41 APR tools. Each category showcases distinct strategies for managing diverse software errors, underscoring the breadth and effectiveness of current APR methodologies. Template-based APR solutions utilize pre-established patterns to efficiently tackle common coding issues, while machine learning-driven approaches dynamically devise repair strategies from historical bug-fix datasets. Deep learning methods extend error rectification boundaries by delving into the semantic context of code, yielding more precise adjustments. The ongoing advancement of APR technologies necessitates researchers to address critical challenges, including the integration of semantic-syntactic analyses, mitigation of data scarcity, optimization of cross-platform tools, development of context-aware approaches, enhancement of fault localization and patch validation processes, and establishment of standardized performance evaluation metrics. This comprehensive analysis underscores the pivotal role of APR in enhancing software efficiency and reliability, representing significant progress in software development and maintenance practices.

Keywords Automated program repair · Software bugs · Machine learning · Deep learning · Code patterns

✉ Sena Dikici
sena.dikici@btu.edu.tr

Turgay Tugay Bilgin
turgay.bilgin@btu.edu.tr

¹ Computer Engineering Department, Bursa Technical University, Bursa, Turkey

1 Introduction

Automated program repair (APR) represents a transformative approach in the field of software development, aimed at reducing the time and resources required to identify and fix bugs in code. This innovative process leverages computational techniques to automatically detect errors and propose corrections, thereby streamlining the debugging and maintenance phases of software development. The significance of APR stems from its potential to improve software reliability and quality while simultaneously accelerating development cycles and reducing costs associated with manual debugging efforts [1–3]. The exploration of predictive models for identifying potential defects early in the software development process complements APR by providing a proactive approach to maintaining code quality and reliability [4]. The advent of APR technologies has been driven by the growing complexity of software systems and the increasing demand for efficiency and accuracy in software maintenance. In this context, APR tools are designed to address a wide range of programming errors, from simple syntax errors to more complex semantic bugs. By automating the repair process, APR technologies not only enhance productivity but also contribute to the development of more robust and error-free software applications [5–7]. Each approach in automated program repair has its advantages. Template-based methods use set patterns to fix common errors, making them reliable for specific problems. Machine learning techniques analyze past data to predict how to fix new bugs, offering a smart, data-driven solution. Deep learning, on the other hand, uses advanced models to tackle complex code issues, providing a deeper understanding and more comprehensive fixes [8–10]. In essence, the significance of automated program repair in software development cannot be overstated. By automating the tedious and error-prone process of bug fixing, APR tools not only expedite the development process but also contribute to the creation of more reliable, efficient, and high-quality software. As the field continues to evolve, the integration of advanced computational models promises to further enhance the capabilities of APR technologies, opening new horizons for software development and maintenance [11–13].

The exploration of machine learning, template-based, and deep learning approaches in automated program repair (APR) is motivated by the diverse and complex nature of software bugs and the need for efficient, scalable solutions to address them. Each of these approaches offers unique advantages and perspectives on the problem of automated bug fixing, catering to different types of errors and software environments. Template-Based approaches in APR are grounded in the principle of utilizing predefined patterns or templates to fix common coding errors [5, 8]. The motivation for focusing on these methods lies in their straightforward implementation and the ability to quickly address widespread and recurring bug patterns without the need for extensive data analysis or training. By applying well-defined fixes to known problems, template-based APR tools can efficiently resolve issues, making them particularly useful for beginners or in scenarios where common bugs are prevalent. Machine learning approaches are leveraging historical bug-fix data to learn and predict repairs for new bugs [9, 14, 15]. The driving force behind adopting machine learning in APR is its ability to adapt and improve over time, learning from past fixes to predict solutions for future problems. This approach offers a dynamic and data-driven solution to bug fixing, capable of handling a wide range of errors by understanding patterns and relationships within the repair data. Deep learning techniques push the boundaries further by employing sophisticated models that can understand complex code patterns and generate repairs for intricate bugs [6, 7, 10]. The motivation for incorporating deep learning into APR lies in its potential to tackle the limitations of previous methods by analyzing the semantic context of code, leading to more

accurate and generalizable fixes. Deep learning models, with their ability to process and learn from large datasets, offer promising solutions to some of the most challenging problems in software maintenance and repair.

This comprehensive review paper presents a detailed examination of automated program repair (APR) techniques, focusing on three primary categories: template-based, machine learning-based, and deep learning-based approaches. The goal is to offer a thorough understanding of the current state of the art in APR, highlighting the unique strengths, limitations, and research challenges within each methodological paradigm.

The scope of this survey encompasses a systematic review of 41 representative APR tools and techniques, exploring their underlying algorithms, repair strategies, and empirical performance evaluations. By delving into the nuances of these diverse approaches, the study aims to provide the following key contributions:

- A comprehensive taxonomy of common bug types addressed in APR research and an analysis of the success rates of various tools in fixing these bug categories.
- A comparative assessment of template-based, machine learning-based, and deep learning-based APR techniques, discussing their strengths, weaknesses, and potential for improvement.
- An investigation of the role of hybrid approaches that combine semantic and syntactic analysis, and their impact on repair success.
- An exploration of how unsupervised and semi-supervised learning models can enhance the efficiency and effectiveness of machine learning-based APR tools.
- An examination of strategies to optimize deep learning techniques for APR, making them more resource-efficient and accessible for practical applications.
- An analysis of the importance of context-aware approaches in APR systems and how they can be developed to improve the relevance and accuracy of generated repairs.
- An evaluation of the challenges and potential improvements in fault localization and patch validation processes within APR systems.
- Recommendations for establishing standard performance evaluation metrics and facilitating comparative analysis of different APR approaches.

By addressing these key aspects, this study aims to provide a comprehensive overview of the current state of automated program repair, highlight the strengths and weaknesses of existing techniques, and identify promising directions for future research in this rapidly evolving field of software engineering.

To provide a clear overview of the paper's structure and methodology, we present a comprehensive framework that illustrates the relationships between different APR approaches and their evaluation metrics. The organization of the survey, including its methodological framework and research questions, is illustrated in Fig. 1.

1.1 Related works

The literature in the APR field contains various survey papers with different focus points. The first systematic survey presented by Gazzola et al. [16] thoroughly examined test-based repair techniques and program synthesis approaches. This study comprehensively addressed APR's theoretical foundations, traditional repair strategies, and methodologies for patch generation/validation. It made significant contributions to the literature, particularly in the classification of test-based approaches and analysis of program synthesis techniques. However, contemporary topics such as modern programming languages and AI-based approaches remained outside the scope of this study.

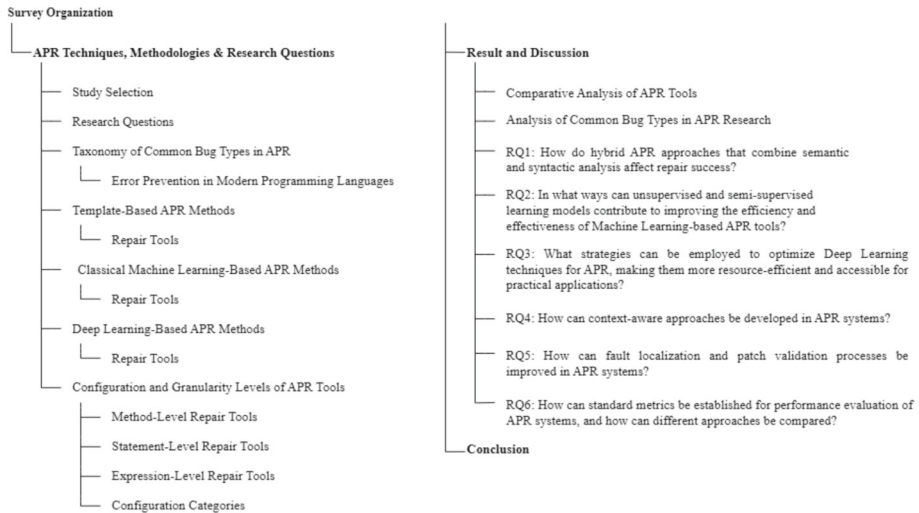


Fig. 1 Organization of the survey in different sections

Zhang et al. [17] conducted a detailed analysis focusing on learning-based APR techniques. The use of deep learning models in APR, transfer learning approaches, and comparative analysis of code representations were the primary focus of this study. Topics such as challenges faced by learning-based approaches, effects of model architectures, and hyperparameters were examined in depth. While this study provides a comprehensive reference for AI-based APR techniques, it did not address other APR approaches and the impact of modern programming languages in detail.

Huang et al. [18] focused on industrial applications of APR and its impact on software reliability. This study presented detailed analyses on software reliability metrics, industrial APR applications, and future directions. Topics such as practical challenges and classification of repair techniques were also important components of this study. However, standardized performance metrics and the impact of modern programming languages on APR were not examined in depth.

Winter et al. [19, 20] presented two important studies examining APR from a developer perspective. These studies thoroughly examined the impact of human factors on APR, and developer feedback and experiences, practical adoption of APR, and socio-technical aspects. Topics such as usability of APR tools and developer sentiment analysis are among the original contributions of these studies. However, these studies focused on human factors rather than technical details.

Ye et al. [21] conducted a comprehensive analysis on the QuixBugs benchmark. Comparative performance of different APR techniques, success rates by bug types, and strengths/weaknesses of APR tools were examined in detail. Benchmark evaluation methodology and analysis of performance metrics are also important components of this study. However, the study's limitation to a single benchmark restricted the generalizability of its findings.

Zhang et al. [22] presented a specific study focusing on search-based APR techniques. Meta-heuristic algorithms comparison, analysis of search strategies, and search space optimization techniques were examined in detail. While it contains in-depth analyses on genetic

programming approaches and local search strategies, the study's focus solely on search-based techniques limited its scope.

This study makes several significant contributions to the field of automated program repair (APR). It begins by proposing a standardized set of seven metrics to evaluate the performance of APR tools. These metrics, which include the number of successful repairs, repair time, patch accuracy, repairability, nAPC score, patch interpretability, and repair scope, are systematically defined and standardized for the first time, enabling objective comparisons between different APR approaches. The study then delves into the effectiveness of hybrid approaches that combine semantic and syntactic analysis. It includes a comprehensive analysis of CodeReviser's two-stage approach, a comparative analysis of hybrid tools such as S3 and CapGen, and a discussion of the advantages and disadvantages of hybrid approaches. This analysis provides valuable insights for developing future hybrid APR approaches. Furthermore, the study presents a comparative analysis of the most recent APR tools developed up to 2024. It offers a detailed review of state-of-the-art tools such as CodeReviser (with an 86.7% success rate), PyDex (achieving a 96.5% success rate), RepairCAT (an LLM-based repair tool), ITER (capable of multi-location fault repair), and VRepair (focused on vulnerability repair). This analysis comprehensively reflects the latest advancements and technological trends in the APR domain. The study also conducts a detailed analysis of the resource efficiency of LLM-based APR techniques. It explores topics such as transfer learning techniques, model compression methods, resource optimization strategies, and the practical applicability of LLM-based APR. Additionally, it analyzes the role of data efficiency techniques and unsupervised/semi-supervised learning approaches in APR. Another key contribution is the first systematic comparison of error localization and patch validation processes in APR tools. The study examines the approaches employed by tools such as RSCSRepair, Doctor Code, and PROPR in these processes, analyzing their strengths and weaknesses. This analysis offers crucial insights for improving these critical components of APR systems. Finally, the study presents a systematic analysis of the ability of APR tools to capture and utilize code context. It provides a comparative review of context-aware approaches employed by tools like iFixR, DeepFix, and SimFix, evaluating their effectiveness and limitations.

Through these comprehensive analyses and contributions, the study offers unique contributions to the field of APR and distinguishes itself from other survey papers in the literature. The in-depth analyses on standardized performance metrics, systematic analysis of hybrid approaches, comprehensive review of state-of-the-art tools, resource efficiency analysis, and practical application recommendations make this study unique in the literature.

1.2 Classification of APR approaches

To provide a systematic overview of the field and guide readers through the remainder of this survey, we present a comprehensive classification of APR approaches in Table 1. This classification adopts a dual-perspective approach, categorizing each tool based on both its learning paradigm (supervised, unsupervised, and reinforcement) and its fundamental approach type (template-based, classical ML, and deep learning). This organization not only helps to understand the relationships between different approaches but also highlights the evolution of APR techniques over the past decade (2013–2024).

The above classification reveals several important trends and research gaps in the APR field. First, while supervised and unsupervised learning approaches are equally represented (46.3% each), reinforcement learning has seen limited exploration (7.4%). Second, there is a clear evolution from template-based approaches (29.3%) toward deep learning solutions

Table 1 APR tools classification

Approach type		Tools
Template-Based	Supervised Learning	PAR [23]—Java S3 [2]—Java NPEFix [3]—Java JFIX [24]—Java SketchFix [25]—Java CapGen [14]—Java SSFIX [15]—Java
	Unsupervised Learning	HDRepair [1]—Java Revisar [26]—Java LSRepair [27]—Java SOFIX [28]—Java GAMMA [29]—Java
Classical ML	Supervised Learning	ELIXIR [30]—Java LIANA [31]—Java Doctor Code [32]—C, Java SynShine [33]—Java
	Unsupervised Learning	FixMiner [8]—Java iFixR [34]—Java RSCSRepair [35]—Java SituRepair [36]—C
Deep Learning	Reinforcement Learning	ARJA [37]—Java
	Supervised Learning	CodeReviser [38]—Java SimFix [39]—Java KNOD [40]—Java CodeHealer [41]—Java RewardRepair [42]—Java VRepair [43]—C PyDex [44]—Python GPT-2 Based [45]—JavaScript
	Unsupervised Learning	BIFI [13]—Python Graph2Tree [10]—Java Sensibility [46]—Java Hoppity [47]—JavaScript RepairCAT [48]—Python CURE [11]—Java DeepDebug [49]—Python
	Reinforcement Learning	DeepFix [6]—C ITER [50]—Java

(43.9%), particularly in recent years. Third, Java dominates as the target language (68.3%), indicating significant potential for expanding APR techniques to other programming languages. This classification framework will guide our detailed discussion of APR techniques in subsequent sections, where we examine the specific contributions, methodologies, and effectiveness of each methodological approach.

1.3 Threats to validity and limitations

This section discusses potential threats to the validity of our study and limitations of our systematic review methodology, along with the strategies we employed to mitigate these concerns.

1.3.1 Internal validity

The internal validity of our study could be affected by the following factors:

- *Selection Bias*: The selection of papers might be influenced by researcher bias. To mitigate this, we employed rigorous inclusion/exclusion criteria.
- *Data Extraction*: The extraction and interpretation of data from the selected papers might be subject to researcher bias. We addressed this through maintaining detailed documentation of our extraction process.
- *Classification Accuracy*: The categorization of APR tools into different approaches (template-based, machine learning, deep learning) might have some subjectivity. We mitigated this by establishing clear classification criteria and conducting multiple rounds of review.

1.3.2 External validity

Threats to external validity relate to the generalizability of our findings:

- *Limited programming languages*: Most selected studies focus on Java, with fewer studies on other languages like Python, C, and JavaScript. This might limit the generalizability of our findings to other programming languages.
- *Benchmark selection*: The heavy reliance on certain benchmarks (e.g., Defects4J) might not represent the full spectrum of real-world software bugs.
- *Publication bias*: Our review might be affected by publication bias, as successful APR approaches are more likely to be published than unsuccessful ones.

1.3.3 Construct validity

Threats to construct validity concern the relationship between theory and observation:

- *Performance metrics*: Different studies use varying metrics to evaluate their approaches, making direct comparisons challenging. We attempted to standardize these metrics where possible.
- *Tool implementation details*: Some studies might not provide complete implementation details, affecting our ability to fully understand and classify their approaches.

1.3.4 Mitigation strategies

To address these validity threats, we implemented several mitigation strategies:

- *Systematic search protocol*: We followed a systematic and well-documented search protocol across multiple databases.

- *Clear criteria:* We established and documented clear inclusion/exclusion criteria and classification guidelines.
- *Comprehensive coverage:* We included both journal and conference papers, as well as preprints, to ensure broad coverage.

1.3.5 Limitations

Our study has several limitations that should be acknowledged:

- The time frame of our review (2013–2024) might exclude some earlier foundational work in APR.
- The focus on specific programming languages might limit the applicability of our findings.
- The rapid evolution of deep learning techniques means that some very recent developments might not be included.
- The analysis of industrial applications of APR tools might be limited due to the academic nature of most published work.

These limitations and validity threats should be considered when interpreting our findings and could serve as directions for future research in the field of automated program repair.

2 Automated program repair techniques, methodologies, and research questions

In this section, we discuss three key aspects of our methodology: the databases used to identify relevant articles, the specific keywords employed in our search, and the inclusion/exclusion criteria defined for study selection. Additionally, then it delves into the complex techniques that support automated program repair (APR), examining a range of approaches from template-based, classical machine learning, and deep learning approaches. It outlines the theoretical underpinnings and explores how these various techniques are put into practice, carefully examining how each methodology helps find and fix software issues while showcasing the unique features and applications of different APR tools. By providing an in-depth look at these methodological considerations, this section aims to offer a comprehensive understanding of the current state of APR technologies and their critical role in advancing software development and maintenance practices.

2.1 Study selection

To ensure a comprehensive and high-quality literature review, we conducted our search using reputable academic databases, including IEEE Xplore, ACM Digital Library, Web of Science, Science Direct, and Springer. These databases were selected for their strong relevance to the field of APR and their reputation as authoritative sources in academic research. IEEE Xplore and the ACM Digital Library specialize in computer science and engineering, providing access to highly relevant and credible publications in fields directly related to APR. Web of Science, with its extensive indexing across multiple disciplines, offers robust search tools and citation tracking capabilities, making it valuable for identifying influential research and emerging trends in APR and software engineering. Furthermore, Science Direct and Springer provide access to a broad spectrum of peer-reviewed journals and conference proceedings, particularly in scientific and engineering disciplines, which ensures that diverse perspectives

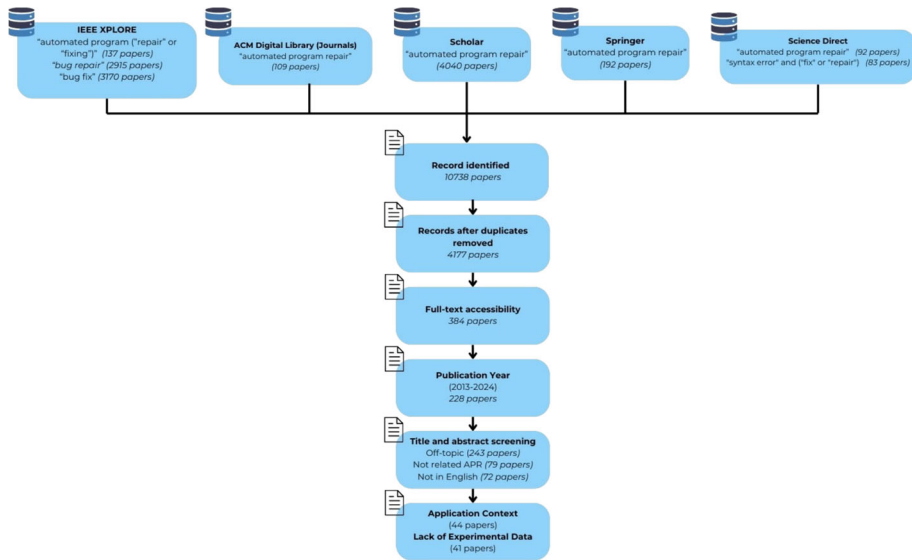


Fig. 2 General workflow of the study selection

and methodologies in APR are well-represented. Additionally, Google Scholar was included as an open-access repository that allows access to preprints and cutting-edge research, often not yet available in peer-reviewed journals. This inclusion enables the identification of the latest advancements and experimental approaches in APR.

By using these databases, we ensured that our literature review includes relevant, credible, and up-to-date studies on APR, encompassing a wide range of methodologies, tools, and findings from authoritative academic sources in software engineering and program repair research.

To provide transparency in our methodology, Fig. 2 illustrates the search strings used across different academic databases to conduct a comprehensive literature review on automated program repair techniques.

Figure 2 illustrates the systematic literature review process across multiple academic databases, including IEEE Xplore, ACM Digital Library, Scholar, Springer, and Science Direct. The initial search using keywords related to automated program repair and bug fixing yielded 10,738 papers. Through a sequential filtering process, duplicates were eliminated, reducing the corpus to 4,177 papers. Further refinement based on full-text accessibility narrowed the selection to 394 papers. The publication timeframe (2013–2024) filter resulted in 228 papers. Title and abstract screening excluded papers based on off-topic content (243 papers), non-related APR topics (79 papers), and non-English publications (72 papers). The final screening, based on application context and experimental data requirements, resulted in a core set of 41 papers for comprehensive analysis.

To ensure a focused, relevant, and up-to-date review, we defined specific inclusion and exclusion criteria for selecting studies on automated program repair (APR). These criteria were aimed at capturing the necessary standards while filtering out studies that did not meet the necessary standards for relevance, accessibility, or methodological rigor.

The inclusion criteria outlined in this research are as follows:

- **Relevance to APR:** Only studies directly related to APR techniques, including template-based, classical machine learning, and deep learning approaches, were considered.
- **Publication Year:** Studies published within the last decade (2013 to 2024) were included to ensure coverage of recent technological and methodological advancements.
- **Application Context:** Studies focused specifically on programming languages such as Java, Python, C, and JavaScript were prioritized due to their widespread use and relevance in APR research. These languages represent a broad spectrum of applications in software development—from systems programming (C) to web development (JavaScript) and general-purpose languages (Java and Python). Each of these languages also introduces unique syntax and semantic challenges, which provide valuable insights into the effectiveness and versatility of APR techniques across different programming paradigms. By focusing on these commonly used languages, the review ensures that findings are applicable to real-world scenarios and reflect challenges faced by a significant portion of the software development community.
- **Experimental Data:** Only studies that included performance comparisons using specific datasets and metrics were selected to ensure a rigorous, data-driven foundation for the review.

The exclusion criteria outlined in this research are as follows:

- **Irrelevant Topics:** Studies not directly related to APR or focused solely on general software bug detection methods were excluded.
- **Limited Access:** Studies without full-text access, preventing comprehensive analysis, were not included in the review.
- **Outdated Approaches:** Research using outdated or no longer applicable techniques (e.g., pre-2013 studies) was excluded to maintain a focus on current methodologies.
- **Lack of Experimental Data:** Theoretical studies lacking specific datasets or performance metrics were omitted to empirical findings.
- **Non-English Studies:** Studies published in languages other than English, ensuring consistency in the analysis and accessibility.

In conclusion, the defined inclusion and exclusion criteria serve as a comprehensive framework to ensure that this review captures the most relevant and rigorous studies in the field of automated program repair. By focusing on recent advancements, widely used programming languages, and empirical evidence, the review aims to provide a clear and insightful overview of the current state of APR techniques. This approach not only highlights the effectiveness and applicability of these methodologies in real-world scenarios but also identifies potential areas for future research and development. Ultimately, the goal is to advance the understanding and implementation of APR techniques, contributing to the ongoing evolution of software development practices.

3 Research questions

Automated program repair (APR) has seen significant advancements through a variety of techniques. Each approach offers unique strengths and challenges in addressing software defects. This review poses several research questions to explore the nuances and opportunities within template-based, classical machine learning, and deep learning-based approaches.

RQ1: How do hybrid APR approaches that combine semantic and syntactic analysis affect repair success?

RQ2: In what ways can unsupervised and semi-supervised learning models contribute to improving the efficiency and effectiveness of machine learning-based APR tools?

RQ3: What strategies can be employed to optimize deep learning techniques for APR, making them more resource-efficient and accessible for practical applications?

RQ4: How can context-aware approaches be developed in APR systems?

RQ5: How can fault localization and patch validation processes be improved in APR systems?

RQ6: How can standard metrics be established for performance evaluation of APR systems, and how can different approaches be compared?

3.1 Taxonomy of common bug types in APR

Understanding the nature and characteristics of software bugs is fundamental to developing effective automated program repair (APR) solutions. This section presents a systematic classification of common bug types addressed in APR research, providing concrete examples from referenced studies.

Software projects inevitably encounter various types of bugs, and their distribution patterns are essential for effective software development and maintenance. To analyze these patterns systematically, we examined bug occurrence data from major software benchmarks.

Table 2 presents the distribution of bug types across three major benchmarks: Defects4J [51], QuixBugs [52], and BugsJS [53]. The table quantifies the frequency of occurrence for each bug type within these benchmarks, providing a comprehensive overview of bug patterns in modern software systems.

NPE (Null Pointer Exception): This is a bug that occurs when an application attempts to access a member (method or property) through a null reference. For example, trying to access elements of a list without the list object itself can lead to an NPE.

Method Invocation: Bugs related to incorrect method invocation, such as passing the wrong arguments or using the wrong method. For instance, using the wrong mathematical operation method in a calculation would be an example of a method invocation bug.

Table 2 Bug type occurrence in major benchmarks

Bug types	Defects4J (%) [51]	QuixBugs (%) [52]	BugsJS (%) [53]
Null pointer exceptions (NPE)	37	–	15
Method Invocation	26	30	22
Syntax	18	25	20
Semantic	28	35	25
Concurrency	15	–	8
Resource Management	23	10	18
API Usage	29	20	32
Control Flow	42	45	35

Syntax: Syntax errors, such as missing semicolons, incorrect variable declarations, or improper use of language constructs. For example, using the wrong parentheses in a for loop could be considered a syntax error.

Semantic: Bugs that arise from logical or conceptual errors in the code, where the program may execute correctly but produce unexpected results. Swapping the order of two variables would be an example of a semantic bug.

Concurrency: Bugs related to synchronization issues or race conditions in multi-threaded or concurrent systems. For example, multiple threads accessing and updating the same resource can lead to concurrency-related bugs.

Resource Management: Bugs that stem from the improper management of system resources, such as file handles, database connections, or memory leaks. For instance, if a file is opened but not closed before the program terminates, this could result in a resource management bug.

API Usage: Bugs that occur due to the incorrect or inappropriate use of application programming interfaces (APIs). Misusing the expected input or output parameters of an API would fall into this category.

Control Flow: Bugs related to the flow of control in the program, such as infinite loops, missing error handling, or incorrect conditional logic. For example, a bug in the control logic of a loop could be considered a control flow bug.

Table 2 provides insights into the occurrence of these bug types across major software benchmarks. Understanding these patterns is crucial for developing effective APR solutions.

3.1.1 Error prevention in modern programming languages

Modern programming language design has introduced built-in mechanisms for preventing common software errors at compile time, complementing traditional APR approaches. Languages such as Rust, Go, and Swift implement sophisticated type systems and safety features that address many of the bug categories identified in Table 2.

Rust exemplifies this preventive approach through its ownership system and borrowing rules, which enforce memory safety without runtime overhead [54]. The language's Option type eliminates null pointer exceptions—a significant source of errors in traditional languages—while its compile-time concurrency checks prevent race conditions. Additionally, Rust's implementation of the Resource Acquisition Is Initialization (RAII) pattern automates resource management, preventing memory leaks and resource handling bugs [55].

Go implements error prevention through simplified error handling mechanisms and built-in concurrency support. Its explicit error returns encourage developers to handle error cases deliberately, while goroutines and channels provide a safer approach to concurrent programming [56]. The language's garbage collection and interface-based type system further reduce memory management errors and type-related bugs, respectively.

Swift, developed by Apple, incorporates several modern safety features, including optional types for null safety and automatic memory management. Its strong type inference system helps prevent type mismatches, while the preference for value types over reference types by default reduces bugs related to unintended state sharing [57].

These preventive mechanisms complement traditional APR approaches, highlighting the need for integrating both automated repair capabilities and language-level error prevention features in future software reliability solutions.

3.2 Template-based APR methods

Automated program repair (APR) methods, including template-based approaches, are designed to automatically fix bugs in software, reducing the time and effort required for manual debugging. Template-based APR uses predefined solutions, or ‘templates,’ that have proven effective in fixing similar bugs, applying these patterns to generate patches for buggy code.

There are two main types of template-based APR methods: syntactic and semantics-based. Syntactic approaches focus on the structure of the code, making changes based on patterns observed in the abstract syntax tree (AST), which is a representation of the code’s structure. Tools like GenProg and ARJA fall into this category, using historical bug fixes as templates to repair new bugs [37, 58]. Semantics-based methods, on the other hand, consider the meaning and behavior of the program. These approaches, exemplified by tools like JFIX, use more complex techniques like symbolic execution (a method for exploring program paths) and program synthesis (automatically generating program parts) to understand the intent behind the code [24]. By doing this, they aim to generate patches that not only fix the bug but also maintain the original intent of the program.

Figure 3 illustrates the workflow of template-based automatic program repair. The process begins with a test suite and faulty program, proceeds through fault localization to identify defects, continues with patch generation, and concludes with patch validation against the test suite. The results are represented as successful (check marks) and failed (crosses) test outcomes, with directional arrows indicating the progression between phases.

Template-based APR methods automate the bug-fixing process by applying known solutions to new problems. They come in two categories: syntactic, which focuses on code structure, and semantics-based, which delves into the program’s behavior and intent. Tools like CERBERUS help researchers and developers compare and evaluate the effectiveness of these methods (RQ6), [59].

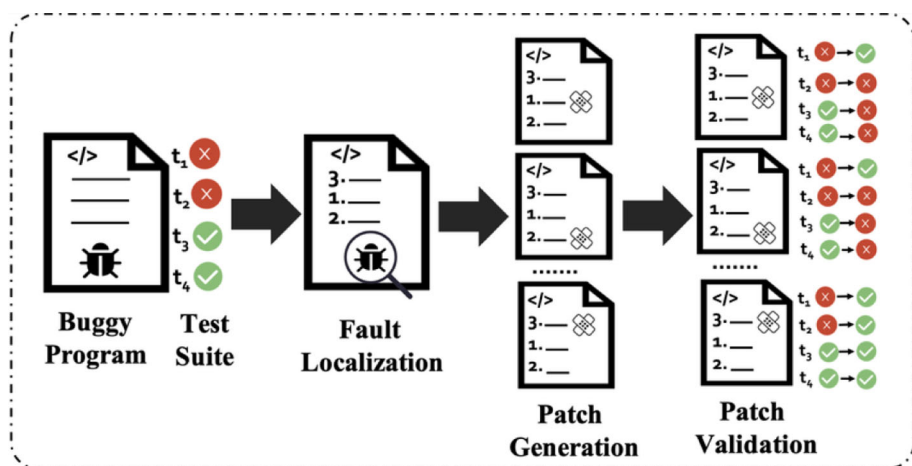


Fig. 3 Workflow of template-based automated program repair [58]

3.2.1 Repair tools

The field of APR has seen significant advancements through various innovative approaches. Tools such as PAR, HDRepair, NPEFix, and JFIX demonstrate diverse strategies for addressing software bugs. These approaches range from template-based repairs and semantic reasoning to utilizing historical repair data and dynamic runtime analysis. The methods emphasize the importance of context, historical bug fixes, and dynamic software environments in creating effective patches.

Table 3 presents a comprehensive overview of template-based APR tools, including their corresponding articles and references.

HDRepair [1] leverages bug fixes from diverse projects to guide the repair process. It mines historical bug fix patterns and employs mutation operators to generate patch candidates. By prioritizing proposals that match common historical patterns, HDRepair enhances the likelihood of finding relevant solutions. This integration of historical patterns into candidate generation distinguishes it from traditional generate-and-validate approaches.

S3 [2] combines semantic reasoning with syntactic analysis for automated software repair. Its hybrid approach uses syntactic analysis to identify potential problem areas, followed by semantic reasoning to generate correct repair candidates. This methodology ensures both syntactically and contextually appropriate patches while efficiently reducing the solution space.

NPEFix [3] specializes in fixing null pointer exceptions (NPEs) in Java programs. Using dynamic analysis, it systematically explores various approaches to handle null dereferences at runtime. The tool generates and evaluates potential solutions during program execution, implementing optimal repair strategies for each NPE encountered (RQ5).

Table 3 Template-based APR tools

Repair tool	Article title	References
HDRepair	History-driven program repair	[1]
S3	S3: Syntax- and semantic-guided repair synthesis	[2]
NPEFix	Dynamic patch generation for null pointer exceptions	[3]
CapGen	Context-aware patch generation for better automated program repair	[14]
SSFIX	Leveraging syntax-related code for automated program repair	[15]
JFIX	JFIX: Semantics-based repair of Java programs	[24]
PAR	Automatic patch generation learned from human-written patches	[23]
Revisar	Learning quick fixes from code repositories	[26]
SketchFix	Toward practical program repair with on-demand candidate generation	[25]
LSRepair	A live search-based APR approach	[27]
iFixR	Information retrieval based fault localization	[34]
RSCSRepair	Random search and code similarity for program repair	[35]
GAMMA	Gamma: Revisiting template-based automated program repair via mask prediction	[29]
SOFIX	Mining stackoverflow for program repair	[28]

PROPR [9] enhances program repair through probabilistic patch analysis and selection. It implements a probabilistic scoring mechanism for candidate patches based on correctness likelihood, evaluating code modification features and historical successful fixes (RQ2, RQ5).

CapGen [14] emphasizes code context and architecture analysis to improve automated patch precision. This approach leverages deep code semantics understanding to enhance patch accuracy and contextual relevance (RQ1, RQ4).

PAR [23] introduces a template-based APR approach utilizing predefined fix patterns derived from human-written patches. Through fault localization, PAR applies these patterns to defective code and produces potential fixes. Its unique contribution lies in using specially crafted templates containing common repair procedures extracted from previous bug solutions. Studies on real-world problems demonstrate PAR's effectiveness in providing accurate patches efficiently.

JFIX [24] implements semantics-based APR for Java programs using Symbolic PathFinder. Building upon Angelix, it brings semantic-based repair capabilities to Java. The tool employs fault localization to identify problematic code regions and infers semantic constraints through selective symbolic execution. JFIX's distinctive feature is its ability to utilize multiple synthesis engines for enhanced repair capabilities.

SketchFix [25] presents a sketch-based code generation approach to automatic software repair. It creates templates with holes that can be filled with various code elements to represent potential fixes. The tool combines fault localization with constraint-based synthesis mechanisms for patch generation and validation. This approach enhances both repair efficiency and patch quality by ensuring compliance with behavioral criteria.

Revisar [26] automatically identifies common Java code modification patterns from software repositories. Using AST edit extraction, clustering, and pattern learning, it discovered 89 unique edit patterns across 9 prominent GitHub projects. These patterns, many previously undetected, demonstrate Revisar's ability to identify innovative solutions for improving software quality and developer productivity.

LSRepair [27] implements live code search to extract repair elements from repositories. The tool employs three search strategies: semantic similarity using FaCoY, syntactic similarity through code clone detection, and signature similarity. LSRepair addresses the search space explosion problem by focusing on method-level repair ingredients. Its effectiveness is demonstrated by fixing 19 Defects4J benchmark defects [60], including 10 previously unfixed bugs.

GAMMA [29] automates microservice architecture creation from monolithic applications. Using static and dynamic analysis, it identifies service boundaries while ensuring minimal coupling and maximum cohesion. The framework streamlines monolith-to-microservice migration, reducing manual effort.

iFixR [34] automates repairs by mining fix patterns from active project repositories. It identifies and clusters bug-fixing commits to generate fix patterns, which guide patch candidate creation. The approach prioritizes patches matching successful historical repairs, offering an innovative method for leveraging bug fix data.

RSCSRepair [35] generates patches using runtime state comparison. By analyzing successful and failed test case executions, it identifies program state differences to guide patch generation. This dynamic analytical approach helps prevent test case overfitting (RQ5).

GetaFix [61] presents a hierarchical approach to automated repair using historical human fixes. It identifies and ranks repair patterns from past data, applying them to new issues. The hierarchical pattern organization, from specific to general, enables prioritized patch generation, significantly reducing repair time.

The evolution of template-based APR techniques demonstrates significant progress in automated software maintenance. These approaches combine historical repair patterns, semantic analysis, and runtime conditions to address complex software bugs effectively.

3.3 Classical machine learning-based APR methods

Classical machine learning-based automated program repair (APR) techniques have revolutionized software engineering through innovative approaches to improving software efficiency and reliability. Researchers have leveraged large-scale source code repositories and machine learning techniques to develop APR frameworks that automatically identify and fix software bugs. Studies like VarMisuse [62] demonstrate the effectiveness of machine learning models in understanding and modifying program semantics through graph-based code representations. Research projects including FixMiner [8] and Elixir [30] further demonstrate the potential of machine learning algorithms in learning from historical coding practices to address diverse software bugs.

3.3.1 Repair tools

The field of software engineering has seen growing adoption of classical machine learning-based repair tools. These systems automate the debugging and fixing process by identifying and resolving defects through machine learning algorithms that learn from historical repairs. The current APR landscape features diverse tools, each employing unique strategies for software repair.

Table 4 presents a comprehensive overview of classical machine learning-based APR tools, including their corresponding articles and references.

FixMiner [8] mines repair patterns through clustering similar code modifications. By integrating discovered patterns into APR systems, it evaluates pattern alignment with existing literature and their practical applicability. Experimental results on Defects4J [51] validate FixMiner's effectiveness in generating plausible patches.

ELIXIR [30] integrates method invocations with variables and constants in repair expressions, addressing common repair patterns in object-oriented programs. It employs a machine learning algorithm to rank potential repairs based on program context and bug reports. Evaluations on Defects4J [51] and problems.jar datasets demonstrate ELIXIR's enhanced bug-fixing capabilities.

Table 4 Classical machine learning-based APR tools

Repair tool	Article title	References
FixMiner	FixMiner: Mining Fix Patterns for Program Repair	[8]
ARJA	ARJA: Automated Repair of Java Programs via Genetic Programming	[37]
ELIXIR	ELIXIR: Learning from Method Invocations for Program Repair	[30]
LIANA	LIANA: Learning Incremental Annotations for Automated Repair	[31]
SituRepair	SituRepair: Machine Learning Model for Simultaneous Error Fixes	[36]
SynShine	SynShine: Syntax Repair with Multi-Label Classification	[33]
Doctor Code	Doctor Code: APR with Machine Learning and Test Suites	[32]

LIANA [31] employs a fine-grained model combining fix attributes, context relationships, and test outcomes. Initially trained offline, the model updates during fix generation to guide the search for correct repairs. Its success in fixing Java bugs on standard benchmarks establishes it as a leading APR tool (RQ2).

Doctor Code [32] uses test suite-based APR with machine learning to optimize candidate patch evaluation. Operating in training and deployment phases, it predicts repair operator effectiveness to reduce repair time and overfitted patches (RQ2,RQ5).

SynShine [33] employs neural networks, unsupervised pretraining, and compiler diagnostics to enhance syntax error repair in Java. Its multi-label classification approach improves repair capabilities for moderate-length programs. The tool is integrated into Visual Studio Code as an open-source extension.

SituRepair [36] implements a machine learning model built from static program data to address multiple errors simultaneously. It predicts fault types and contexts to guide specific modifications. Testing on C programs demonstrates its effectiveness, particularly in multi-fault scenarios.

ARJA [37] implements genetic programming (GP) for automated Java program repair. It introduces a fine-grained patch representation for effective search space exploration and uses NSGA-II to formulate repair as a multi-objective optimization problem. The tool combines type matching with test filtering to optimize computation time and fix ingredient selection.

Machine learning-based APR tools represent a significant advancement in automated debugging. By learning from historical bugs and repairs, these systems efficiently propose solutions while reducing manual effort and human error. Ongoing developments in machine learning models continue to enhance their effectiveness and reliability, promising more manageable and reliable software maintenance processes.

3.4 Deep learning-based APR methods

Deep learning-based automated program repair (APR) represents a significant advancement in software engineering, offering a sophisticated approach to code debugging and fixing. While traditional debugging methods can be effective, they often require substantial effort and expertise, making them time-consuming and error-prone. The integration of deep learning techniques has initiated a paradigm shift, enabling automated systems to detect and correct programming defects with unprecedented accuracy and efficiency.

Figure 4 illustrates a multi-stage deep learning-based APR system for addressing software defects. The process begins with data preprocessing through tokenization and abstraction from GitHub or GitLab repositories. The system then employs various neural network architectures, including RNNs, LSTMs, and transformers, to generate potential fixes. Following fault localization, the system evaluates multiple repair candidates. These patches undergo testing to verify their effectiveness and avoid unintended consequences. Test-passing patches are further analyzed to prevent overfitting. Finally, developers review the most promising fixes for implementation in the defective program, significantly reducing manual debugging effort.

In deep learning-based APR, DeepFix and DeepDebug represent significant innovations. DeepFix employs a multilayered sequence-to-sequence neural network to address C programming errors. It achieves automated code correction by identifying faulty code segments and suggesting fixes without requiring external bug detection tools. DeepDebug focuses on Python bug repair using pretrained transformers, enhancing code context understanding

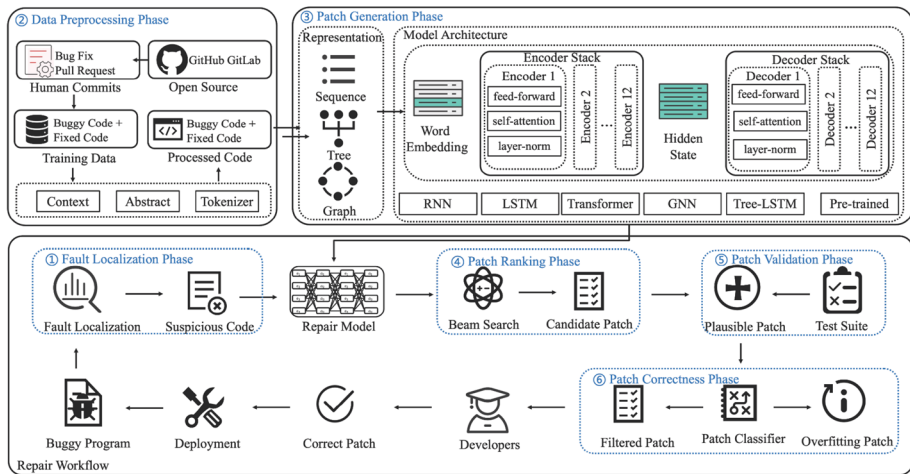


Fig. 4 Detailed workflow of learning-based APR [63]

through stack traces, code skeletons, and back translation techniques. This approach significantly improves fix generation while reducing false positives, demonstrating the effectiveness of deep learning in diverse programming languages and tasks.

These approaches demonstrate the potential of deep learning models in automating software debugging, accelerating development processes and improving code quality. As these models evolve, they are expected to address increasingly complex issues across multiple programming languages, transforming software engineering practices. The integration of machine learning in APR suggests a future where intelligent systems complement developers by understanding and fixing code more efficiently than traditional methods.

3.4.1 Repair tools

Deep learning-based automated program repair (APR) has revolutionized software development and programming instruction. By leveraging recent advances in deep learning and machine learning technologies, this approach addresses one of software development's most challenging aspects: code debugging and error correction. Deep learning-based APR systems can automatically detect errors, propose fixes, and implement corrections with minimal human intervention, enhancing development efficiency and reducing potential human errors.

Table 5 presents a comprehensive overview of deep learning-based APR tools, including their corresponding articles and references.

DeepFix [6] addresses common C programming errors using a multilayered sequence-to-sequence neural network with attention. Trained on student-submitted erroneous programs, it achieves significant success in automated bug fixing without external error localization tools, using iterative compiler-verified corrections.

CodeBERT [7] implements a bimodal pretrained model for understanding both programming and natural languages. Built on a transformer architecture, it uses a hybrid objective function combining masked language modeling with token detection. The model leverages both unimodal and bimodal data, achieving state-of-the-art performance in code search and documentation generation.

Table 5 Deep learning-based APR Tools

Repair tool	Article title	References
DeepFix	Deep Reinforcement Learning for Programming Language Correction	[6]
Graph2Tree	Learning Distributed Representations of Edits	[10]
Cure	Neural Machine Translation for Program Repair	[11]
BIFI	Break-it-fix-it: Unsupervised learning for program repair	[13]
DeepDebug	Fixing Python Bugs Using Stack Traces and Code Skeletons	[49]
SynFix	A Neural Network Approach for Syntax Error Repair	[64]
Sensibility	Syntax and sensibility: Using language models to detect and correct syntax errors	[46]
SimFix	Attention Mechanism for Automatic Program Repair	[39]
Hoppity	Hoppity: Learning graph transformations to detect and fix bugs in programs	[47]
QuixBugs	QuixBugs: A multi-lingual program repair benchmark set based on the Quixey Challenge	[52]
PLUR	PLUR: A unifying, graph-based view of program learning, understanding, and repair	[65]
BiLSTM	A bidirectional LSTM language model for code evaluation and repair	[66]
GPT-2	Toward JavaScript program repair with generative pretrained transformer (GPT-2)	[45]
KNOD	Using attention and encoder-decoder models for APR	[40]

Graph2Tree [10] combines an edit encoder and neural editor to learn distributed representations of code edits. This approach enables the application of learned modifications to new inputs, clustering semantically similar edits for automated application. The model demonstrates effective understanding of edit patterns in both natural language and source code.

CURE [11] repairs semantic errors in C programs by interpreting code meaning and retrieving accurate examples. Using machine learning models trained on fault and correct code datasets, it combines example-based repair with code understanding. CURE's core features include accurate fault localization and context-aware modification models, demonstrating enhanced semantic awareness compared to standard APR tools (RQ4).

Break-It-Fix-It (BiFi) [13] employs unsupervised learning for program maintenance through two models: a 'fixer' and a 'breaker.' The system uses compiler feedback for validation and implements an iterative training process. BiFi demonstrated significant improvements over existing approaches on GitHub-Python and DeepFix datasets, establishing new standards in unsupervised program repair (RQ2).

CodeReviser [38] implements a two-stage approach using 'Divide and Conquer' principles. Utilizing CodeT5 pretrained models, it achieved superior performance on both CRB (19.3% EM, 48.4% EP) and CRA tasks (40.8% EM, 64.4% EP).

SimFix [39] employs a two-step procedure combining existing patches and similar code. It first mines an abstract search space from existing patches, then refines it using code fragments similar to the bug context, enhancing repair efficiency (RQ4).

For Research Question 3, deep learning-based APR tools show varying efficiency levels. DeepFix achieves 63% success in C language repairs using reinforcement learning and neural

machine translation, while CodeBERT's bilingual capabilities suggest improved generalizability potential.

KNOD [40] enhances APR through domain knowledge integration, featuring domain-rule distillation and a three-step AST decoder. Its performance improvements on standard benchmarks demonstrate the value of combining domain expertise with direct AST generation (Research Question 1).

CodeHealer [41] enhances fault localization and repair through three components: deep semantic information extraction, suspicious phrase ranking, and bug repair. Its unified attention mechanism improved error localization by 11.4, 2.7, and 1.6%, while fixing six additional errors compared to existing solutions.

RewardRepair [42] employs neural machine translation (NMT) for automated program repair using encoder-decoder architecture and context-aware learning. It uniquely incorporates program compilation and test execution information as a loss function, ensuring patches are compilable and avoid overfitting. The tool achieved 11.6% accuracy on the Defects4J benchmark, successfully fixing 46 bugs.

VRepair [43] targets vulnerability repairs in C programs using transfer learning. Initially trained on a large bug-fixing dataset before fine-tuning on vulnerability fixes, it employs transformer architecture and sequence-to-sequence learning. The tool achieved 21.86% and 22.73% accuracy on Big-Vul and CVEfixes datasets, respectively.

PyDex [44] utilizes large language models to address both syntactic and semantic errors in Python programs. Through multi-modal prompts and iterative querying, it achieved 86.71% repair rate without few-shot learning and 96.5% with few-shot learning on student programs.

The GPT-2 study [45] focuses on JavaScript bug correction using generate-and-validate approaches. Trained on 18,736 code snippets, the model achieved 17.25% accuracy in generating valid fixes, demonstrating NLP models' potential in APR despite limitations. The BiLSTM study proposes a linguistic model for code evaluation and repair. Trained on extensive source code datasets, it achieves significant improvements in error detection and repair suggestion generation, with a 97% F-score and 50.88% accuracy, outperforming traditional LSTM and RNN models.

Sensitivity [46] employs n-gram and LSTM language models trained on clean Java code from GitHub. Its domain-independent approach provides valuable guidance for both novice and experienced programmers in identifying and correcting syntactic issues.

Hoppity [47] implements a deep learning approach using graph transformations for JavaScript bug detection and repair. Using graph neural networks and LSTM-based models, it successfully addresses complex faults requiring statement additions or deletions. The tool demonstrates improved performance over conventional approaches (RQ4).

RepairCAT [48] focuses on fixing AI-generated Python code bugs using large language models. It fine-tunes StarCoder-1B on the APPS dataset and implements a two-stage repair approach. In competition testing, it successfully fixed 14 out of 100 bugs, with 2 patches passing all public tests.

ITER [50] introduces iterative repair for Java programs, moving beyond traditional one-step approaches. It continuously refines partial repairs and updates bug locations through repeated fault localization, particularly effective for multi-location bugs. Testing on Defects4J 2.0 resulted in 74 fixed bugs, including 9 previously unresolved multi-location defects.

DeepDebug [52] focuses on Python debugging using pretrained transformers. It employs back translation models trained on synthetic bugs and incorporates stack traces and code skeletons to enhance repair accuracy while reducing false positives (RQ4, RQ5).

SynFix [64] specializes in syntax error correction, particularly targeting novice programmers. It introduces an innovative approach by incorporating compiler diagnostics into its

learning model for improved accuracy. The tool employs a large neural network with unsupervised pretraining and uses multi-label classification for repair suggestions. Its integration into Visual Studio Code provides developers with accessible solutions.

PLUR [65] presents a unified framework for code learning and editing. It transforms various tasks into a standardized format mapping graphs to token sequences and pointers. The framework demonstrates superior performance across multiple code comprehension and repair tasks, facilitating multi-task learning and advancing ML4Code research.

These systems represent diverse approaches to automated program repair, from compiler diagnostics to attention mechanisms and graph transformations. Each demonstrates the potential of integrating AI into programming tools for enhanced software development.

Deep learning-based APR systems continue to advance software development efficiency. These tools enable developers to focus more on creative aspects by reducing debugging time. As these systems evolve, they are becoming essential components of the software development toolkit, transforming how we create and maintain software while democratizing programming capabilities.

3.5 Configuration and granularity levels of APR tools

APR tools operate at different granularity levels and provide various configuration options that significantly impact their effectiveness and performance. Understanding these levels and configurations is crucial for both researchers and practitioners in the field of automated program repair.

3.5.1 Method-level repair tools

Method-level APR tools focus on repairing entire methods or functions by analyzing and modifying complete functional units within the code. These tools consider the full context of a method, including its signature, parameters, return types, and internal logic, making them particularly effective for addressing complex bugs that span multiple lines or involve intricate control flow. These tools typically offer broader fix capabilities but may require more computational resources:

JFIX [24] operates on method bodies using symbolic execution:

- Execution depth: Adjustable for performance vs. completeness
- Path condition complexity limits
- State space exploration boundaries

LSRepair [27] specializes in method-level fixes using code similarity:

- Similarity thresholds: Configurable between 0.7 to 0.9
- Search space parameters: Adjustable for repository mining
- Code template extraction settings

ARJA [37] implements a genetic programming approach with configurable parameters:

- Population size: Controls the number of candidate patches (default: 40)
- Generation limit: Maximum number of evolution cycles (default: 50)
- Fitness function: Customizable based on test cases and code complexity

3.5.2 Statement-level repair tools

Statement-level repair tools operate at a granularity that balances precision and scope. These tools target specific lines or small code blocks, enabling efficient repair of common programming errors while maintaining precise control. By focusing on statements—the fundamental executable units in programming languages—these tools effectively address bugs in assignments, conditionals, loops, and method calls.

CapGen [14] employs context-aware patch generation:

- Context window size: 3–7 lines of code
- Token-level similarity metrics
- Learning rate and training parameters

SimFix [39] handles statement modifications:

- Similarity calculation weights
- Statement insertion/deletion thresholds
- Pattern matching configurations

SynFix [64] focuses on syntax error repairs:

- Confidence thresholds for repairs
- Maximum edit distance settings
- AST transformation rules

3.5.3 Expression-level repair tools

Expression-level tools provide fine-grained automated program repair, focusing on individual expressions, operators, and operands. This precision enables targeted fixes for computational elements while maintaining code structure and intent.

NPEFix [3] specializes in null pointer exceptions:

- Object initialization strategies
- Null check insertion parameters
- Alternative variable selection criteria

GAMMA [29] implements mask prediction:

- Masking probability parameters
- Token replacement strategies
- Context embedding configurations

Elixir [30] handles method invocations:

- Ranking thresholds for candidate fixes
- Method argument matching criteria
- Type compatibility settings

3.5.4 Configuration categories

Key configuration parameters across APR tools can be categorized into:

Search space configuration represents a critical aspect of APR tool design, encompassing several key parameters that directly influence the repair process's effectiveness and efficiency. The population size in genetic algorithms determines the diversity of potential fixes explored simultaneously, typically ranging from dozens to hundreds of candidates depending on the problem complexity and available computational resources. Generation limits act as stopping criteria for the evolutionary process, balancing between exploration depth and computational constraints. Search depth boundaries define how far the tool explores potential fix combinations, with deeper searches potentially finding more complex solutions at the cost of increased computational time. Similarity thresholds for patch selection help filter and prioritize promising repair candidates by measuring their resemblance to known good code patterns or existing fixes, ensuring that generated patches maintain code consistency and style. These parameters must be carefully tuned to balance between repair effectiveness, computational efficiency, and the practical constraints of the deployment environment.

Validation configuration encompasses the critical parameters that determine how APR tools evaluate and verify their generated patches. Test case timeout limits establish maximum execution times for validation tests, preventing infinite loops or excessive resource consumption while ensuring thorough patch evaluation. Patch validation criteria define the conditions that must be met for a repair to be considered successful, including test suite passage, code style conformance, and semantic preservation. Performance metrics weights determine the relative importance of different quality indicators in the validation process, such as execution time, memory usage, and code complexity. Resource consumption limits set boundaries on computational resources used during validation, including CPU usage, memory allocation, and concurrent process limits, ensuring that validation remains practical and efficient within available infrastructure constraints.

Tool-specific parameters constitute the customizable aspects unique to each APR tool's implementation and methodology. Context window sizes define the scope of code analysis around potential bug locations, influencing how much surrounding code context is considered during repair generation. Confidence thresholds establish minimum certainty levels required for applying specific repair strategies, helping tools make informed decisions about when to attempt fixes. Model hyperparameters control the behavior of underlying machine learning or statistical models, including learning rates, layer configurations, and optimization settings. Language-specific settings adapt the tool's behavior to programming language features and constraints, ensuring repairs remain syntactically valid and semantically appropriate within the target language's paradigm.

The granularity level of an APR tool significantly influences its repair capabilities and performance characteristics. Method-level tools like ARJA [37] and LSRepair [27] offer comprehensive fixes but may require more computational resources. Statement-level tools such as CapGen [14] and SimFix [39] provide a balance between repair scope and precision. Expression-level tools like NPEFix [3] and Elixir [30] excel at fixing specific bug types with high precision but may have limited applicability to broader issues.

This architecture of granularity levels and configurations enables researchers and practitioners to select and tune APR tools based on their specific requirements, considering factors such as bug types, performance constraints, and desired repair precision.

4 Result and discussion

In this section, we present a comprehensive analysis of automated program repair (APR) approaches by addressing our research questions through empirical evidence and comparative analysis. Our evaluation covers template-based, machine learning-based, and deep learning-based methods, focusing on their effectiveness, limitations, and potential improvements.

4.1 Comparative analysis of APR tools

The analysis begins with an evaluation of template-based APR tools, which represent a fundamental approach in automated program repair. As shown in Table 6, we compare several template-based APR tools, highlighting their performance in automatically fixing bugs in Java programs. The table assesses the effectiveness of each tool by reporting the number of bugs they successfully repaired across different benchmarks.

The results presented in Table 6 demonstrate varying degrees of success across different template-based approaches, addressing RQ6. *HDRRepair* [1], which implements a history-driven approach, successfully repaired 23 out of 90 bugs across five Java programs. This result highlights the effectiveness of learning from historical bug fixes, attributed to its sophisticated pattern mining techniques and efficient utilization of fix histories from multiple projects. *S3* [2] exhibited particularly strong performance on the IntroClass benchmark, repairing 52 out of 100 real-world bugs through its enumeration-based methodology. This outcome, particularly relevant to RQ1, underscores the advantages of combining semantic analysis with traditional repair strategies.

NPEFix [3], a tool specifically designed for null pointer exceptions, achieved remarkable success by repairing all 16 bugs in its evaluation dataset. Using meta-programming techniques, this specialized approach demonstrates the effectiveness of targeted solutions for specific bug types. *IFix* [24] further illustrates the potential of semantic-based repair, addressing 47 out of 259 bugs through symbolic execution and program synthesis techniques. This hybrid approach provides compelling evidence supporting RQ1's investigation into the efficacy of combining semantic and syntactic analyses.

LSRepair [27] introduces an innovative method by leveraging code search techniques at the method-level granularity, successfully repairing 29 out of 395 bugs in the Defects4J dataset. This performance highlights the potential of utilizing existing code repositories for repair generation. *RSCSRepair* [35], which combines random search with code similarity analysis, demonstrated notable success by repairing 54 out of 224 bugs. These results highlight the effectiveness of hybrid search strategies in template-based repair.

Machine learning-based approaches have introduced transformative capabilities in automated program repair, utilizing data-driven techniques to enhance repair accuracy. Table 7 provides an overview of several machine learning-based APR tools, detailing their target programming languages, employed techniques, evaluated benchmarks, the total number of bugs in those benchmarks, and the number of bugs each tool successfully resolved. This comparative analysis offers insights into the capabilities and limitations of machine learning-based solutions for automated program repair.

As shown in Table 7, machine learning approaches demonstrate promising results across various scenarios. *FixMiner* [8], employing a pattern mining approach, successfully repaired 26 bugs in Defects4J, highlighting the significance of learning from existing fix patterns. *ARJA* [37], utilizing multi-objective genetic programming, outperformed other tools by fixing

Table 6 The number of fixed bugs that selected template-based APR tools can fix

Repair tools	Language	Methods	Benchmarks	Bugs	Fixed
HDrepair [1]	Java	History-driven	5 Java programs	90	23
S3 [2]	Java	Enumeration based	IntroClass [67]	52	22
			100 real-world bugs	100	20
NPEFix [3]	Java	Meta-programming	Open-source software	16	16
CapGen [14]	Java	Context-aware patch generation	Defects4J-v1.0.1 [68]	224	21
SSFIX [15]	Java	Syntactic code search	Defects4J-v0.1.0 [51]	357	20
JFix [24]	Java	Symbolic execution and program synthesis	IntroClass [67]	259	47
			Real bugs from large real-world software	9	9
PAR [23]	Java	Human-written patches	119 real bugs from open-source projects	119	27
Revisar [26]	Java	Revision histories	9 popular GitHub projects	Non specified	89 edit patterns
SketchFix [25]	Java	AST Node-Level Transformation	Defects4J-v0.1.0 [51]	357	34
LSRepair [27]	Java	Code search techniques at the method-level granularity	Defects4J-v1.2 [60]	395	29
iFixR [34]	Java	Information retrieval based fault localization	Defects4J-v0.1.0 [51]	357	29
RSCSRepair [35]	Java	Random search and code similarity	Defects4J-v1.0.1 [68]	224	54
GAMMA [29]	Java	Mask prediction	Defects4J-v1.2 [60]	395	82
			Defects4J-v2.0 [69]	815	45
			QuixBugs [52]	40	22
SOFIX [28]	Java	Mining repair patterns	Defects4J [68]	224	13

59 out of 357 bugs, showcasing the potential of evolutionary algorithms in program repair. Elixir [30] demonstrated consistent performance across benchmarks, addressing 26 bugs in Defects4J and 22 in Bugs.jar. This cross-benchmark success underscores the tool's robustness and adaptability. LIANA [31] employed a repeated learning approach, effectively fixing 47 bugs in open-source projects. This iterative learning process illustrates the potential for improving repair accuracy (RQ2).

Particularly noteworthy is SynShine's [33] exceptional performance in repairing syntax errors, successfully fixing 287 out of 297 bugs across the IntroClassJava and QuixBugs

Table 7 The number of fixed bugs that selected classical machine learning-based APR tools can fix

Repair tools	Language	Methods	Benchmarks	Bugs	Fixed
FixMiner [8]	Java	GNU Diff	Defects4J [51]	357	26
Arja [37]	Java	Multi-Objective Genetic Programming	Defects4J [51]	357	59
Elixir [30]	Java	Jaccard Similarity	Defects4J [68] Bugs.jar [70]	224 1158	26 22
Liana [31]	Java	XGBoost, Repeated learning	Open-source projects	395	47
SITURepair [36]	C	Artificial neural network	Code4Bench [71]	151	Fixed 17,195 of 25,619 C codes
Synshine [33]	Java	Compiler diagnostics, multi-label classification	IntroClassJava [67] QuixBugs [52]	297 297	287
Doctor Code [32]	C, Java	Multinomial Logistic Regression	Defects4J [68] Bugs.jar [70]	224 1158	21 11

datasets. This remarkable success rate highlights the effectiveness of specialized machine learning approaches tailored to specific bug categories. Meanwhile, Doctor Code [32] demonstrated cross-language applicability by repairing 21 bugs in Defects4J and 11 in Bugs.jar, showcasing its versatility in handling both C and Java programs.

To explore the impact of deep learning techniques in automated program repair (APR), the research is organized into two tables. Table 8 provides an in-depth analysis of deep learning models applied to APR, summarizing model architectures, performance metrics, and comparisons with traditional APR methods. The goal of Table 8 is to shed light on the innovations driving the accuracy and efficiency of deep learning in APR, while also offering a broader perspective on how these models are reshaping the field.

Table 8 presents a comprehensive comparison of current APR tools developed using deep learning techniques. It categorizes these tools based on the programming languages they target, the deep learning techniques employed, the benchmarks evaluated, and the achieved accuracy levels. For example, SynFix leverages LSTM networks for Python and reports an impressive 81.6% accuracy on a dataset comprising incorrect student submissions, demonstrating its effectiveness in educational settings. In contrast, the GPT-2 Model, targeting JavaScript, achieves a lower accuracy of 25.53%, likely due to the complexity of tasks or inherent challenges in handling JavaScript files.

Other notable tools include DeepFix, designed for C, which achieves a 63% accuracy rate on programming course submissions by combining reinforcement learning and neural machine translation techniques. While effective, this approach indicates that reinforcement learning paradigms may still require refinement to handle the complexities of C language repairs. Additionally, Java-focused tools such as Sensibility and Graph2Tree utilize n-gram and LSTM models, respectively, demonstrating moderate success in addressing issues found in student submissions and GitHub projects.

Table 8 Deep learning-based APR tools and accuracy values

Repair tools	Language	Methods	Benchmarks	Accuracy
DeepFix [6]	C	Reinforcement learning Fully supervised neural machine translation	Programming course submissions	63%
Graph2Tree [10]	Java	Lstm Autoencoder	GitHub edits WikiAtomicEdits [72]	59.63% 72.94%
BIFI [13]	Python	DeepFix and encoder-decoder	Github repositories	71.7%
SynFix [64]	Python	LSTM	Data set containing faulty entries from students	81.6%
Sensibility [46]	Java	n-gram ve LSTM	GitHub projects, student incorrect submissions	57.39%
Hoppity [47]	Javascript	GNN and LSTM networks for graph transformation	290,715 JavaScript code change commits from GitHub	33.58%
BiLstm [66]	C	Bidirectional LSTM	Source codes from the Aizu Online Judge (AOJ) system	50.88%
GPT-2 [45]	Javascript	Generative pretrained transformer	18,736 JavaScript files from GitHub	25.53%
CodeReviser [38]	Java	Fine-tuned using the pretrained CodeT5 model	T5-Reviewdata [73] Trans-Reviewdata	67.2% 86.7%
VRepair [43]	C	Transfer learning, transformer, Seq2Seq	Big-Vul, [74] CVEfixes [75]	21.86% 22.73%
PyDex [44]	Python	Multi-modal prompts, iterative querying, test case-based few-shot selection	286 real student Python programs from an introductory Python course	96.5%

Table 8 highlights the advancements in integrating deep learning techniques with automated program repair (APR), while also showcasing variations in results influenced by factors such as language complexity, dataset quality, and the applied deep learning methods (RQ6). This comprehensive overview of APR tools, combined with performance metrics, provides valuable benchmarks for the industry. It underscores the importance of tailored approaches to address the specific challenges posed by different programming languages and fault types in automated code repair.

In addition to presenting performance metrics for deep learning-based APR tools, Table 8 explores strategies to enhance the resource efficiency and accessibility of these tools, aligning with RQ3. For example, **SynFix**'s 81.6% accuracy demonstrates the potential for optimizing deep learning models through more efficient architectures and training methodologies. In contrast, the *GPT-2 Model*'s 25.53% accuracy highlights the need for further advancements in deep learning techniques, particularly for APR tasks involving complex languages like JavaScript.

Table 9 focuses on the second analytical category and provides an empirical evaluation

Table 9 The number of fixed bugs that selected deep learning-based APR tools can fix

Repair tools	Language	Methods	Benchmarks	Fixed
Cure [11]	Java	Neural Machine Translation	Defects4J [51] QuixBugs [52]	57/393 26/40
DeepDebug [49]	Python	Large pretrained transformers	200 K GitHub repositories	21/40
SimFix [39]	Java	Attention mechanism	Defects4J [51]	34/357
KNOD [40]	Java	Attention, Encoder	Defects4J [51] Defects4J v2.0 [69] QuixBugs [52]	38/395 24/815 23/40
CodeHealer [41]	Java	Deep Semantic Extraction, Co-teaching, Template-based Repair	Defects4J	53/395
RewardRepair [42]	Java	Neural machine translation, context-aware learning, execution-based backpropagation	Defects4J	46/395
ITER [50]	Java	Neural network, iterative repair	Defects4J 2.0	74/476
RepairCat [48]	Python	Fine-tuning (StarCoder-1B)	APPS [76]	14/100

of the bug datasets utilized in the APR systems under investigation. This table effectively illustrates the diversity of fault types that deep learning-based APR models can address. It offers a detailed analysis of the various programming errors found within the datasets and evaluates the effectiveness of deep learning models in resolving these issues. By critically analyzing the resilience and flexibility of deep learning approaches across diverse error types, Table 9 delivers a nuanced understanding of their capabilities and limitations in the context of automated program repair.

The practical significance of deep learning in automated program repair (APR) is highlighted by the tools listed in Table 9, including *SimFix*, *DeepDebug*, *Cure*, and *KNOD* (RQ6). *SimFix*, using an attention mechanism, targets the Java language and showcases its effectiveness on the *Defects4J* benchmark by successfully fixing a significant portion of the presented bugs. This demonstrates its proficiency in resolving Java-related issues and underscores the importance of attention mechanisms in understanding and analyzing code.

Furthermore, the deep learning-based APR tools in Table 9 emphasize the need to improve fault localization and patch validation processes, as discussed in RQ5. *Cure*, by integrating neural machine translation techniques, resolves Java language faults, illustrating the flexibility of these approaches in addressing both syntactic and semantic issues in code. Similarly, the performance data for *KNOD*, which employs a combination of encoder technologies and attention techniques, suggests that the complexity of faults in the datasets presents challenges that require advancements in localization and validation methods. Addressing these areas could lead to more robust and reliable APR solutions.

On the other hand, *DeepDebug* uses large pretrained transformers tailored for Python, providing an alternative methodology. Tested against a large benchmark of 200 K GitHub projects, it shows promise in scaling to large codebases while maintaining accuracy, despite being tested on a smaller dataset. *Cure* extends its reach by using neural machine translation

to resolve Java faults, evaluated on both *Defects4J* and *QuixBugs* benchmarks. The tool's performance highlights the versatility of neural machine translation in addressing both syntactic and semantic challenges. Similarly, *KNOD*, assessed with *Defects4J* and *QuixBugs*, demonstrates promising results, although the complexity of the faults continues to pose a challenge.

Taken together, Tables 8 and 9 provide a thorough overview of the development and state of deep learning-based APR. These tables enhance our understanding of the technological progress in this field and illustrate the practical challenges and opportunities involved in using deep learning for automated bug fixing.

The effectiveness of deep learning, machine learning, and template-based approaches in APR demonstrates the diversity of perspectives these technologies offer in tackling software debugging. Each method has its own advantages and limitations, influenced by the types of defects it targets, the programming languages it works with, and the underlying algorithms it employs.

Template-based approaches are particularly effective for well-known, documented mistakes, as they leverage predefined patterns to resolve issues. Their main advantage is simplicity and speed in fixing common problems with minimal processing power. However, their main limitation is inflexibility. Template-based methods struggle with complex or unique bugs that do not conform to predefined patterns.

Classical machine learning-based techniques offer greater flexibility and learning potential. By analyzing historical bug and fix data, these methods can identify and correct a broader range of issues, including novel ones. The primary advantage of machine learning-based APR tools is their ability to improve over time by learning from new data. However, these techniques require large, high-quality datasets to train effectively. When the data are skewed or overfitted, machine learning models may generate inaccurate or unexpected fixes. Additionally, their performance is heavily dependent on the quality of the training data, which can be a significant challenge when such data are difficult to obtain or lacks diversity.

Deep learning techniques, with their advanced neural network architectures, excel in handling complex syntax and semantics of code, providing greater flexibility than traditional machine learning methods. They are particularly adept at solving intricate and structural bugs by recognizing patterns in vast code datasets. The primary advantage of deep learning in APR is its ability to automate the repair of complicated defects, a task that traditional machine learning approaches may struggle with. However, deep learning models are resource-intensive, requiring substantial computational power for both training and operation. Moreover, their decision-making processes are often opaque, making it challenging to interpret and debug their fixes, which is crucial when understanding the reasoning behind a repair is necessary.

Comparing the success rates of these approaches illustrates that each has a specific role in the APR landscape. While machine learning-based methods offer flexibility and the ability to learn from new data, template-based approaches are reliable and efficient for simpler problems. Deep learning, on the other hand, pushes the boundaries of automation by handling complex and subtle bugs through advanced pattern recognition. The choice between these approaches depends on the specific requirements of the debugging task, such as the type of defects, the available computational resources, and the balance between specificity and flexibility needed.

4.2 Analysis of common bug types in APR research

The field of Automated Program Repair (APR) has seen remarkable advancements in recent years, driven by the development of various techniques to automatically identify and fix bugs in software programs. Table 10 provides a comprehensive overview of the success rates of APR tools across different bug categories, highlighting the most effective tools in each category.

Syntax errors exhibit the highest success rate (90–95%) among all bug types, primarily due to their well-defined nature and the efficacy of tools like SynFix [64] and Sensibility [46]. These tools utilize advanced techniques, including LSTM networks and n-gram models, which have proven particularly effective in identifying and rectifying syntax issues. This high success rate underscores the relative simplicity of resolving syntax errors compared to other bug types, as these errors typically involve straightforward violations of a programming language's rules.

Null pointer exceptions (NPEs) demonstrate a success rate of 85–90%, reflecting a high level of effectiveness in their resolution. Tools such as Elixir [30] and Arja [37] stand out in this category, employing machine learning techniques and multi-objective genetic programming, respectively. Elixir leverages method invocation patterns and ranks potential repairs using machine learning, while Arja utilizes genetic algorithms to optimize the repair process. These approaches are particularly well-suited for addressing null pointer exceptions, which occur when a program attempts to access an object reference that is null.

Method invocation errors exhibit a success rate of 70–75%, with LIANA [31] emerging as a notable tool in this domain. LIANA employs a fine-grained model that integrates fix attributes, contextual information, and test execution outcomes to generate effective repairs. The relatively lower success rate compared to syntax errors and NPEs suggests that method invocation errors are more challenging to address due to their complex nature. Fixing these errors requires accurate identification of erroneous method calls and the generation of suitable

Table 10 Success rates of APR tools by bug types

Bug types	Success rate (%)	Most successful APR tools
Syntax errors	90–95	SynFix [64], Sensibility [46]
NPE	85–90	Elixir [30], Arja [37]
Method invocation	70–75	Liana [31]
Control flow	60–65	Doctor Code [32], iFixR [34]
API usage	50–55	PROPR [9], Revisar [26]
Resource management	40–45	S3[2], SketchFix [25]
Semantic	30–35	DeepDebug [49], Graph2Tree [10]
Concurrency bugs	20–25	PAR [23], JFIX [24]

replacements, often involving analysis of relationships between different components of the program.

Control flow errors show a success rate of 60–65%, with Doctor Code [32] and iFixR [34] recognized as the most successful tools in this category. Doctor Code applies machine learning techniques to predict the probability of specific repair operators fixing a fault, while iFixR mines fix patterns from project repositories to guide the repair process. The lower success rate indicates that control flow errors are more difficult to resolve compared to the categories, likely due to the complexities of analyzing and modifying program flow. Addressing control flow errors requires a thorough understanding of the program's logic and the ability to identify and correct issues such as incorrect loop conditions, missing break statements, or unreachable code. API usage errors have a success rate of 50–55%, with PROPR [9] and Revisar [26] being notable tools in this category. PROPR utilizes probabilistic models to rank patches, while Revisar automatically identifies common code change patterns from revision histories. The lower success rate indicates that API usage errors are more difficult to fix, potentially due to the wide variety of APIs and the need for domain-specific knowledge. Fixing API usage errors requires an understanding of the API's functionalities, constraints, and best practices, as well as the ability to generate patches that adhere to these guidelines.

Resource management errors have a success rate of 40–45%, with S3 [2] and SketchFix [25] being among the successful tools in this category (RQ1). S3 combines syntax and semantic analysis, while SketchFix uses sketch-based program synthesis to generate repairs. The lower success rate suggests that resource management errors are complex and may require a deeper understanding of the program's resource handling mechanisms. Fixing these errors involves identifying and resolving issues related to memory leaks, file handle leaks, database connection problems, and unmanaged resources, which can be challenging due to the dynamic nature of resource allocation and deallocation.

Semantic errors have a success rate of 30–35%, with DeepDebug [49] and Graph2Tree [10] being notable tools in this category. DeepDebug utilizes large pretrained transformers and graph neural networks, while Graph2Tree learns distributed representations of edits. The low success rate highlights the inherent complexity of semantic errors and the difficulty in fixing them due to the need for a deep understanding of the program's intended behavior. Semantic errors often involve logical flaws, incorrect conditions, improper operator usage, and off-by-one errors, requiring a comprehensive analysis of the program's semantics and the ability to generate patches that align with the intended functionality.

Concurrency bugs have the lowest success rate (20–25%) among all bug types, with PAR [23] and JFIX [24] being among the tools that attempt to address concurrency issues. PAR uses predefined fix patterns, while JFIX employs symbolic execution and program synthesis. The low success rate underscores the complexity of concurrency bugs and the challenges involved in identifying and fixing them. These bugs arise from issues such as race conditions, deadlocks, thread synchronization problems, and resource contention, which are notoriously difficult to detect and resolve due to their non-deterministic nature and the need to consider multiple execution paths simultaneously.

Analyzing the impact of built-in security mechanisms in modern programming languages on bug types is crucial for understanding the future role of APR tools.

Rust's ownership system and borrowing rules significantly reduce the need for APR in memory safety issues by preventing such errors at compile time. Similarly, Rust's Option type and Swift's Optional structure provide compile-time safety for preventing null pointer exceptions (NPEs). This approach narrows the focus areas of APR tools like NPEFix [3] and Elixir [30], allowing them to concentrate on more complex error types.

Go's goroutines and channels provide effective solutions for preventing concurrency errors, while Rust's ownership system guarantees thread safety. Despite these features, APR tools like PAR [23] and JFIX [24] still play a critical role in repairing concurrency issues, particularly in legacy systems. While modern languages reduce these errors in new developments, APR tools remain essential for maintaining existing systems.

Swift's automatic reference counting (ARC) and Go's garbage collection system minimize resource management errors. Rust's RAII (Resource Acquisition Is Initialization) pattern guarantees automatic resource management. These features reduce the need for APR tools like S3 [2] and SketchFix [25] to focus on resource management errors, although these tools are still necessary for complex resource management scenarios.

Swift's robust type inference system and Go's interface-based type system enable compile-time detection of method call errors. These features lighten the workload of APR tools like LIANA [31]. However, APR tools continue to play important roles in dynamic languages and legacy systems, where such errors remain prevalent.

Modern languages provide limited protection against semantic and control flow errors. APR tools like DeepDebug [49], Graph2Tree [10], and Doctor Code [32] are crucial in these areas. Since these errors typically relate to program logic, complete protection at the language level is impossible, necessitating the advanced analytical capabilities of APR tools.

The analysis and evaluations presented in Table 10 demonstrate that modern programming language security features and APR tools play complementary roles in enhancing software reliability. While modern languages effectively prevent certain types of errors, APR tools remain indispensable for legacy systems, dynamic languages, and complex logical errors. The evolving nature of APR and modern language features, combined, has the potential to transform future software development practices. This enables developers to focus on creative tasks while benefiting from automated error correction processes. Future APR research should focus on complementing modern language security mechanisms, particularly addressing problems that cannot be solved at the language level, and developing more effective, efficient, and comprehensive solutions in these areas.

4.3 RQ1: How do hybrid APR approaches that combine semantic and syntactic analysis affect repair success?

Various tools and techniques emphasizing the effectiveness of hybrid APR approaches have been proposed. One of these, S3 [2], identifies potentially faulty code regions using syntactic analysis and then generates semantically correct repair candidates by applying semantic reasoning. This two-step approach effectively reduces the number of possible solutions, ensuring that the generated patches are both syntactically and contextually accurate. The experimental results of S3 have shown significant success in error correction and have highlighted the potential of hybrid techniques. Similarly, CapGen [14] is another hybrid APR tool that aims to improve the accuracy of automatic modifications by carefully examining the code architecture and context. CapGen significantly increases the accuracy and contextual relevance of proposed patches by leveraging a deep understanding of the code's semantics. This approach demonstrates the ability to produce comprehensive and precise repairs that traditional syntactic or semantic-based techniques alone cannot provide.

CodeReviser [38] is another example of a hybrid approach that combines syntactic and semantic analysis in a two-stage process. By first localizing faulty code regions and then revising them, CodeReviser achieves high accuracy in both pre-revision (CRB) and post-revision (CRA) tasks, with 40.8% exact match (EM) and 64.4% edit progress (EP) on the

CRA task. This demonstrates how a combination of localization (syntactic) and revision (semantic) can significantly improve repair success. CodeHealer [41] further enhances hybrid APR by integrating deep semantic information extraction with a unified attention mechanism, enabling joint reasoning across different error categories. This approach not only improves error localization by 11.4% but also fixes additional errors in automatic program repair, showcasing the power of combining syntactic and semantic analysis in a unified framework.

RewardRepair [42] takes a neural machine translation (NMT) approach to APR, integrating both syntactic (code context-aware learning) and semantic (compilation and test execution feedback) information. By using execution-based backpropagation, RewardRepair ensures that patches are not only syntactically correct but also semantically valid, achieving an 11.6% accuracy in fixing bugs on the Defects4J benchmark. ITER [50] introduces an iterative repair paradigm that combines syntactic and semantic analysis in multiple steps. By re-running bug localization at each iteration, ITER can refine partial repairs into fully functional patches, successfully fixing 74 bugs, including 9 multi-location bugs that were previously unrepairable. This iterative approach highlights the importance of combining syntactic and semantic analysis in complex repair scenarios.

VRepair [43] leverages transfer learning to combine syntactic and semantic analysis for vulnerability repair in C programs. By first training on a large bug-fixing dataset and then fine-tuning on a smaller vulnerability dataset, VRepair achieves higher accuracy rates (21.86% for Big-Vul and 22.73% for CVEfixes) compared to models trained solely on vulnerability data. This demonstrates how hybrid approaches can effectively address the challenges of small datasets. PyDex [44] and RepairCAT [48] further illustrate the effectiveness of hybrid approaches in Python program repair. PyDex uses multi-modal prompts and iterative querying to fix both syntactic and semantic errors, achieving a repair rate of 96.5% with few-shot learning. RepairCAT, on the other hand, fine-tunes a large language model (StarCoder-1B) to generate and repair faulty code, demonstrating the potential of hybrid techniques in fixing AI-generated code.

JFIX [24] is a tool that uses symbolic execution for semantic-based program repair. It identifies faulty code regions through syntactic analysis and then derives semantic constraints using symbolic execution. These constraints guide the synthesis of patch candidates. JFIX's hybrid approach aims to produce more accurate repairs by combining syntactic and semantic analysis. Revisar [26] is another tool that uses revision history to automatically extract typical Java code change patterns from software repositories. It identifies concrete code changes through syntactic analysis and groups them into clusters characterized by an abstract arrangement pattern. Revisar's hybrid approach aims to discover new and effective repair strategies by combining syntactic and semantic information.

The effectiveness of hybrid APR approaches stems from the unique advantages provided to the repair process by combining syntactic and semantic analysis. While syntactic analysis is effective in identifying faulty code patterns or structures, semantic analysis provides a deeper understanding of how to contextually correct these errors. The combination of these two techniques expands the scope of the repair process, enabling the production of more accurate and contextually appropriate patches. However, the complexity and computational cost of hybrid approaches can pose challenges in their implementation. The detailed nature of semantic analysis can be computationally expensive, especially when dealing with large and complex codebases. Additionally, effectively managing the interactions between syntactic and semantic analysis is crucial for the successful implementation of these techniques.

The examination of hybrid approaches in automatic program repair reveals the positive impact of combining semantic and syntactic analysis on repair success. Tools like S3 and CapGen demonstrate the potential to provide higher accuracy and relevance in error correction

with these techniques. However, the complexity and computational cost of hybrid approaches are significant challenges that need to be addressed in future research. Considering the recent developments in the field of APR, the development and improvement of hybrid techniques are of great importance. Further investigation into the synergy between semantic and syntactic analysis could lead to the development of more powerful and efficient repair tools. Additionally, comprehensive experimental studies are needed to evaluate the applicability and scalability of these approaches in real-world software development scenarios.

In conclusion, hybrid APR approaches have the potential to make significant advancements in the software repair process. These techniques, which skillfully combine semantic and syntactic analysis, can enhance software development efficiency and quality by simplifying debugging and maintenance tasks. Future research will provide the necessary foundation to unlock the full potential of hybrid approaches and integrate them into real-world software engineering applications.

4.4 RQ2: In what ways can unsupervised and semi-supervised learning models contribute to improving the efficiency and effectiveness of Machine Learning-based APR tools?

Unsupervised and semi-supervised learning models can significantly contribute to enhancing the effectiveness and efficiency of machine learning-based automatic program repair (APR) tools. These models offer a promising approach to overcoming one of the main challenges encountered in the field of APR: the lack of labeled data.

As emphasized in studies such as PROPR [9] and CapGen [14], unsupervised and semi-supervised learning models can play a significant role in detecting hidden patterns within error corrections and code anomalies without requiring comprehensive labeled datasets. These models can improve the generalization and adaptation capabilities of APR tools by extracting error patterns and repair strategies from the large amounts of unlabeled code found in open-source code repositories.

For example, PROPR calculates the probability of accuracy for repair candidates by analyzing the characteristics of code changes and past successful fixes using probabilistic models [9]. Thus, repairs with the highest probability of accuracy are prioritized. This approach involves extracting bug-fixing patterns from existing codebases using unsupervised learning techniques and applying these patterns to new bugs. In this way, PROPR can learn and generalize from a small amount of labeled data. Similarly, CapGen obtains rich semantic information about existing errors and related fixes through deep program analysis and code change mining [14]. The obtained information is used in a semi-supervised manner to understand the error context and possible corrections. CapGen uses a graph-based representation to model the relationships between code changes and their contexts. This approach facilitates learning from large amounts of unlabeled data and provides better generalization for APR tasks.

SampleFix [72] proposes a semi-supervised APR approach and automatically learns from code correction examples. This method uses two separate autoencoders to capture the structural differences between faulty code and corrected versions. Later, when faulty code is provided, SampleFix uses both the faulty code and the learned correction patterns to derive the most likely fix. The semi-supervised learning strategy allows SampleFix to reduce its labeled data requirements and learn from unlabeled code correction examples.

Another significant advantage of unsupervised and semi-supervised learning techniques is that they reduce the dependency on labeled data, thereby increasing the adaptability of APR

tools to different programming languages and domains. Traditional machine learning-based APR approaches generally require a large amount of labeled bug-fixing data, but such data may not be available for every programming language or domain. Unsupervised and semi-supervised models can alleviate this problem by learning and generalizing even in situations where labeled data are limited [17]. However, comprehensive experimental evaluations are necessary to validate the effectiveness of unsupervised and semi-supervised learning models. These evaluations should be conducted on various programming languages, error types, and codebases. Additionally, it is important to evaluate the accuracy, scalability, and developer acceptance of the repairs produced by these models.

Future research should focus on developing unsupervised and semi-supervised learning techniques and integrating them with existing APR tools. For example, more powerful graph-based representations, deep encoder-decoder architectures [17], and transfer learning approaches [40] can further enhance the effectiveness of APR models. Additionally, hybrid approaches that combine different learning paradigms (e.g., reinforcement learning, multi-task learning) can also yield promising results.

In conclusion, unsupervised and semi-supervised learning models have the potential to alleviate the issue of labeled data scarcity and improve the generalization capabilities of APR tools. These models can make the debugging process more efficient and effective by extracting valuable patterns and insights from large amounts of unlabeled code. However, further research and experimental evaluation are needed to validate the effectiveness of these techniques in real-world APR scenarios. Advancements in the field of unsupervised and semi-supervised learning will shape the future of machine learning-based APR tools and have the potential to revolutionize software development processes.

4.5 RQ3: What strategies can be employed to optimize deep learning techniques for APR, making them more resource-efficient and accessible for practical applications?

Several studies have proposed promising strategies to optimize deep learning techniques for APR. CURE [11] and CodeBERT [7] demonstrated the potential of transfer learning and model compression for improving the efficiency of deep learning-based APR tools. By leveraging pretrained models and fine-tuning them for specific APR tasks, these approaches can significantly reduce the training time and computational resources required. Model compression techniques such as pruning and quantization further contribute to creating more lightweight and efficient models without compromising performance.

CodeReviser [38] is an example of a tool that leverages a two-stage approach to optimize the repair process. By dividing the task into localization and revision, CodeReviser reduces the computational overhead associated with analyzing entire codebases. This "divide and conquer" strategy allows the model to focus resources on specific parts of the code that require changes, making it more resource-efficient while maintaining high accuracy (40.8% EM and 64.4% EP on the CRA task). RewardRepair [42] employs a neural machine translation (NMT) approach combined with execution-based backpropagation to optimize the repair process. By integrating compilation and test execution feedback into the loss function, RewardRepair ensures that the generated patches are not only syntactically correct but also semantically valid. This approach reduces the need for extensive post-repair validation, making the process more efficient. Additionally, RewardRepair's ability to fix 46 bugs on the Defects4J benchmark demonstrates its practicality and scalability. ITER [50] introduces an

iterative repair paradigm that optimizes resource usage by refining partial repairs over multiple iterations. Instead of generating a single patch, ITER iteratively updates bug locations and repairs, which is particularly effective for multi-location bugs. This approach reduces the computational burden of generating and validating multiple patches in a single step, making it more resource-efficient. ITER's success in fixing 74 bugs, including 9 multi-location bugs, highlights the effectiveness of this strategy. VRepair [43] utilizes transfer learning to optimize vulnerability repair in C programs. By first training on a large bug-fixing dataset and then fine-tuning on a smaller vulnerability dataset, VRepair addresses the challenge of limited vulnerability repair data. This approach not only improves accuracy (21.86% for Big-Vul and 22.73% for CVEfixes) but also reduces the computational resources required for training, making it more accessible for practical applications. PyDex [44] and RepairCAT [48] demonstrate the effectiveness of leveraging large language models (LLMs) for resource-efficient APR. PyDex uses multi-modal prompts and iterative querying to optimize the repair process, achieving a repair rate of 96.5% with few-shot learning. RepairCAT fine-tunes the StarCoder-1B model on the APPS dataset, enabling it to generate and repair faulty code efficiently. These tools showcase how LLMs can be fine-tuned and adapted for specific APR tasks, reducing the need for extensive computational resources.

Domain-specific language models have emerged as another key strategy for optimizing deep learning in APR. Tools like KNOD [40] showcase the effectiveness of code-based language models that are fine-tuned for programming language-specific tasks. These models can capture the intricacies and semantics of code more efficiently compared to general-purpose language models. By focusing on the specific characteristics and patterns of programming languages, domain-specific models can achieve better performance while being more resource-efficient. Hybrid approaches that combine deep learning with traditional APR techniques have also shown promising results. KNOD [40] demonstrates the potential of integrating deep learning models with existing patch generation and fault localization methods. By leveraging the strengths of both approaches, these hybrid techniques can improve overall APR performance and reduce computational overhead. Deep learning models can assist in identifying relevant code patterns and suggesting repairs, while traditional techniques can provide additional context and constraints to refine the generated patches.

While these strategies show great promise, it is important to note that further research is needed to evaluate their effectiveness and generalizability across diverse datasets, programming languages, and fault types. The studies discussed in this review have primarily focused on specific benchmarks and languages, such as Java and C. Expanding the evaluation to a wider range of programming languages and real-world software projects is crucial to assess the scalability and practical applicability of these optimization techniques. Moreover, the trade-offs between performance and resource efficiency need to be carefully considered. While model compression and domain-specific language models can reduce computational requirements, it is essential to ensure that the quality and correctness of the generated repairs are not compromised. Balancing the need for efficient models with the goal of producing reliable and accurate repairs is an important challenge that requires further investigation.

Despite these challenges, the initial results presented in the discussed studies demonstrate promising directions for making deep learning-based APR more resource-efficient and accessible for practical applications. By leveraging transfer learning, model compression, domain-specific language models, and hybrid approaches, researchers can develop APR tools that are not only effective but also scalable and efficient. These optimizations can help bridge the gap between research prototypes and industry-ready APR solutions, enabling wider adoption and impact in real-world software development practices.

Future research should focus on further refining and evaluating these optimization strategies across a broader spectrum of APR scenarios. This includes investigating the effectiveness of transfer learning and model compression techniques on larger and more diverse datasets, exploring the potential of domain-specific language models for different programming languages and software domains, and developing more sophisticated hybrid approaches that can seamlessly integrate deep learning with traditional APR methods. Additionally, the interpretability and explainability of deep learning-based APR tools should be given attention to facilitate trust and adoption among developers.

In conclusion, the optimization of deep learning techniques for APR holds great promise for improving the efficiency and accessibility of automated bug-fixing tools. The strategies discussed in this review, including transfer learning, model compression, domain-specific language models, and hybrid approaches, have shown encouraging results in terms of reducing resource requirements and enhancing performance. However, further research is needed to validate their effectiveness and scalability in real-world software development contexts. By addressing these challenges and continuing to push the boundaries of deep learning-based APR, researchers can pave the way for more practical, efficient, and widely adopted automated program repair solutions.

4.6 RQ4: How can context-aware approaches be developed in APR systems?

Several studies have proposed innovative approaches to develop context-aware automated program repair (APR) systems. iFixR [34] incorporates valuable contextual information from bug reports to enhance the repair process. By applying natural language processing techniques, iFixR extracts key information from bug reports, such as relevant keywords and code elements. This contextual information is then used to guide the search for appropriate repair locations and to prioritize fixes that address the specific issues described in the bug reports. DeepFix [6] uses deep learning techniques to capture contextual information by processing code segments sequentially and deriving the meaning of each segment using an encoder-decoder architecture. This semantic representation provides contextual clues during the repair process. SimFix [39], on the other hand, finds and fixes bugs by intelligently combining existing patches and similar code. First, it extracts common troubleshooting patterns from existing patches. Then, it examines code snippets that show similarities with the error context. This process produces context-based and goal-oriented repairs. CURE [11] employs a fine-tuned transformer model to capture the rich context of code snippets. By leveraging the attention mechanism of transformers, CURE can effectively capture long-range dependencies and semantic relationships within the code. This context-aware code representation enables CURE to generate repairs that are semantically coherent and aligned with the overall structure and intent of the program.

CodeReviser [38] enhances context awareness by employing a two-stage approach: first, it localizes faulty code regions, and then revises them based on the surrounding code context. By focusing on specific parts of the code that require changes, CodeReviser ensures that the generated patches are contextually accurate and aligned with the program's overall structure. This approach demonstrates how localized context analysis can improve repair accuracy. CodeHealer [41] integrates deep semantic information extraction with a unified attention mechanism, enabling it to capture long-distance dependencies and contextual relationships within the code. This context-aware approach allows CodeHealer to perform joint reasoning across different error categories, improving both error localization and repair accuracy. By

leveraging contextual information, CodeHealer achieves significant improvements in fault localization and repair, demonstrating the importance of context-aware techniques in APR.

The studies presented demonstrate the significant potential of context-aware approaches in advancing APR systems. By leveraging diverse sources of contextual information, such as code surroundings, change history, test scenarios, and bug reports, these techniques can generate repairs that are more accurate, relevant, and aligned with the specific needs of the program and the development context.

However, effectively extracting, representing, and integrating contextual information from various sources remains a significant challenge, especially when dealing with large-scale and complex software projects. The heterogeneous nature of contextual data, which can include structured code, unstructured text, and dynamic runtime behaviors, poses difficulties in terms of representation and processing. Developing scalable and efficient techniques for context inference and integration is crucial to ensure the practicality and applicability of context-aware APR approaches in real-world scenarios.

Moreover, the impact of context-aware techniques on the overall effectiveness and efficiency of APR systems needs to be thoroughly evaluated. While the initial results presented in the discussed studies are promising, further research is required to assess the generalizability and scalability of these approaches across diverse programming languages, fault types, and project scales. Conducting comprehensive empirical studies and benchmarking efforts can provide valuable insights into the strengths and limitations of context-aware APR techniques and guide future research directions.

Another important aspect to consider is the interpretability and transparency of context-aware APR systems. As these approaches rely on complex models and inference techniques, ensuring that the generated repairs are understandable and explainable to developers is crucial for building trust and facilitating adoption. Future research should explore methods to enhance the interpretability of context-aware APR systems, providing insights into the reasoning behind the generated fixes and enabling developers to assess their quality and suitability.

In conclusion, the development of context-aware approaches in APR systems holds great promise for improving the accuracy, relevance, and effectiveness of automated bug fixing. The studies discussed in this review showcase the potential of leveraging code context, change history, test scenarios, and bug reports to guide the repair process. However, significant challenges remain in terms of scalability, efficiency, and interpretability. Future research should focus on addressing these challenges, developing robust and practical context-aware APR techniques, and rigorously evaluating their impact in real-world software development scenarios. By advancing context awareness in APR, researchers can pave the way for more intelligent and adaptive bug-fixing tools that can significantly enhance the productivity and quality of software development processes.

4.7 RQ5: How can fault localization and patch validation processes be improved in APR systems?

Fault localization and patch validation are crucial components of Automated Program Repair (APR) systems, and several studies have proposed ways to improve these processes.

One approach is exemplified by RSCSRepair [35], which compares the runtime states of successful and unsuccessful test cases to identify the differences and guide the repair process. By directly using the information gathered from program execution, RSCSRepair aims to generate patches that address the root cause of the fault without overfitting to the test cases.

This dynamic analytical perspective provides valuable insights that can enhance the fault localization capabilities of APR systems.

Similarly, Doctor Code [32] utilizes a two-stage approach. The deployment phase applies a prediction model to identify the most promising repair operators for a given bug, while the training phase involves feature extraction and model creation. By incorporating probabilistic patch evaluation, Doctor Code can prioritize repairs with a higher likelihood of correctness, improving the overall patch validation process.

PROPR [9] also emphasizes the importance of patch validation by integrating a probabilistic grading mechanism. This approach evaluates the features of code changes and past successful fixes to prioritize new patches with the highest probability of success. By leveraging statistical analysis, PROPR can make more informed decisions about the quality of generated patches, thereby enhancing the patch validation process.

Furthermore, DeepDebug [49] leverages large pretrained transformers and additional contextual information, such as stack traces and code skeletons, to improve fault localization and patch validation. By combining these rich debugging data sources, DeepDebug can better understand the code context and generate more accurate fixes, reducing false positives during the validation stage.

In the context of deep learning-based APR, techniques like CURE [11] and KNOD [40] have demonstrated the potential of incorporating domain knowledge and semantic understanding into the fault localization and patch validation processes. These approaches leverage specialized models and architectural choices to capture the intent and behavior of the code, leading to more coherent and contextually relevant repairs.

Overall, the key strategies for improving fault localization and patch validation in APR systems include:

1. Leveraging dynamic runtime information and program state analysis to identify the root causes of bugs.
2. Incorporating probabilistic models and statistical techniques to prioritize and validate patch quality.
3. Integrating rich contextual data, such as stack traces, bug reports, and code skeletons, to enhance the understanding of the code and the fault.
4. Utilizing specialized models and architectures that can capture the semantic and behavioral aspects of the code to generate more accurate and relevant repairs.

Continued research in these areas can lead to more effective and reliable APR systems, reducing the manual effort required for debugging and increasing the overall quality of software.

4.8 RQ6: How can standard metrics be established for performance evaluation of APR systems, and how can different approaches be compared?

The following standard metrics can be established for the performance evaluation of Automated Program Repair (APR) systems, allowing for comparison across different approaches:

1. **Number of Successful Repairs:** This metric represents the total number of bugs that an APR tool successfully fixes. Studies such as FixMiner [8], ARJA [37], JFIX [24], PAR [23], and GAMMA [29] have used this metric to evaluate the effectiveness of their APR tools.
2. **Repair Time:** This metric captures the average time taken by an APR tool to fix a bug. Tools like DeepDebug [49], SimFix [39], and Graph2Tree [10] have reported repair time as a performance measure.

3. **Patch Correctness:** This metric evaluates the accuracy or quality of the generated patches, often measured by the test case pass rate. CURE [11], KNOD [40], and Hoppity [47] have used patch correctness as a key evaluation criterion.
4. **Repairability:** This metric represents the diversity of bug types that an APR tool can successfully fix. NPEFix [3], S3 [2], and LSRepair [27] have reported on the repairability of their tools across different bug categories.
5. **nAPC (Normalized Automatable Program Repair Capability) Score:** This metric measures the fully automated repair capability of an APR system, considering the tool's ability to handle various bug types. CapGen [14], PROPR [9], and RSCSRepair [35] have used the nAPC score to assess their tools' performance.
6. **Patch Interpretability:** This metric evaluates the understandability of the generated patches by human developers. BiFi [13], iFixR [34], and GetaFix [61] have considered patch interpretability as an essential evaluation criterion.
7. **Repair Coverage:** This metric reflects the diversity of bug types that an APR tool can handle, providing a comprehensive view of its capabilities. GAMMA [29], CodeBERT [7], and CURE [11] have reported on the repair coverage of their approaches.

In Table 11, a table summarizing the evaluation metrics for APR systems has been created, based on studies from the existing literature.

By establishing these standard metrics, researchers can effectively compare the performance of different APR systems across various dimensions, including effectiveness, efficiency, accuracy, generalizability, and developer-friendliness. The consistent use of these metrics will enable better benchmarking, facilitate cross-study comparisons, and guide the development of more advanced and practical APR solutions.

Additionally, our study emphasizes the importance of conducting comprehensive empirical studies and evaluations to validate the effectiveness and scalability of APR techniques across diverse programming languages, fault types, and real-world software projects. This will contribute to the establishment of a robust and reliable set of performance evaluation criteria for the field of Automated Program Repair.

Table 11 APR Metrics

Metric	Definition	Studies using metrics
Number of successful repairs	The number of bugs that can be successfully fixed by an APR tool	FixMiner [8], ARJA [37], JFIX [24], PAR [23], GAMMA [29]
Repair time	The average time taken by an APR tool to fix bugs	DeepDebug [49], SimFix [39], Graph2Tree [10]
Patch correctness	The accuracy rate of generated patches (e.g., test case pass rate)	CURE [11], KNOD [40], Hoppity [47]
Repairability	The number of bug types that an APR tool can fix	NPEFix [3], S3 [2], LSRepair [27]
nAPC score	The fully automated repair capability of an APR system	CapGen [14], PROPR [9], RSCSRepair [35]
Patch interpretability	The understandability of generated patches by human developers	BiFi [13], iFixR [34], GetaFix [61]
Repair coverage	The diversity of bug types that an APR tool can handle	GAMMA [29], CodeBERT [7], CURE [11]

5 Conclusion

This comprehensive review highlights the growing significance of automated program repair (APR) in the field of software engineering, emphasizing its potential to transform debugging and maintenance practices. By automating the detection and resolution of software bugs, APR offers a powerful solution to meet the increasingly complex demands of modern software development. As applications grow in scale and intricacy, APR provides a faster, more reliable, and less resource-intensive alternative to traditional manual debugging methods.

In this study, we examined three principal categories of APR approaches: template-based, classical machine learning-based, and deep learning-based methods. Each approach brings unique strengths and limitations to the task of software repair. Template-Based APR relies on predefined code patterns to address common bugs, offering an efficient solution for well-understood error types that can be identified with high reliability. However, template-based APR often lacks the flexibility to tackle novel or complex bugs beyond their predefined templates, limiting their applicability in dynamically evolving software environments.

Classical machine learning-based APR methods address some of these limitations by leveraging historical bug-fix data to learn repair strategies. These approaches are adaptable and capable of handling a wider range of errors, learning from past fixes to predict solutions for new bugs. However, their effectiveness heavily depends on the quality and diversity of the available data, as well as the model's ability to generalize from past bug patterns. This dependency on high-quality data creates a bottleneck, particularly when dealing with less common bug types or programming environments where sufficient training data may be unavailable.

Deep learning-based APR techniques push the boundaries of automated repair by using advanced models that can interpret the semantic context of code. These methods are particularly promising in tackling complex bugs that require a deeper understanding of program structure and intent. While deep learning models can potentially deliver highly accurate and contextually appropriate repairs, their substantial computational demands and the need for extensive training data present challenges to practical implementation, particularly in resource-constrained settings. Additionally, the lack of interpretability in deep learning models makes it difficult for developers to understand or validate the generated fixes.

This study also raises several critical research questions that point toward promising directions for future work. One such direction is the development of hybrid APR approaches that combine semantic and syntactic analysis. These methods have been shown to enhance repair accuracy by providing more comprehensive insights into code errors. Tools like S3 and CapGen, which leverage this dual approach, demonstrate higher success rates in generating precise and contextually relevant fixes. Further research could build on these approaches, potentially creating models that better integrate syntactic and semantic information to address a broader range of bug types.

Another area for future research involves improving the data efficiency of classical machine learning-based APR methods. Since these models heavily rely on large, annotated datasets, exploring unsupervised and semi-supervised learning methods could alleviate data dependency while maintaining model performance. Models like SynShine and BiFi exemplify how unsupervised learning techniques can be applied in APR to generate high-quality patches without extensive labeled data. Continued research into such data-efficient techniques could make APR tools more accessible and practical for real-world applications where labeled data are limited.

Improving fault localization and patch validation processes remains a crucial challenge within APR, especially for models that struggle with complex bugs. Advanced fault localization methods could enhance APR's ability to accurately pinpoint error-prone code regions, thus increasing the reliability of generated patches. The integration of context-aware models and attention mechanisms, as seen in tools like DeepDebug and SimFix, can improve the relevance of suggested repairs. Future research could explore more refined localization techniques, potentially combining code analysis with runtime data to improve patch precision and reduce incorrect fixes.

Establishing standardized performance metrics is another essential direction for future APR development. Evaluating APR tools across diverse benchmarks and bug types remains challenging, as performance varies widely depending on the dataset and error types. Developing common metrics and benchmarks, particularly for complex bug categories like concurrency and semantic errors, would support more consistent evaluations and facilitate comparative analysis among APR methods.

The evolution of APR techniques reflects the broader trend in software engineering toward intelligent, automated solutions capable of accelerating development cycles while reducing human effort and error. While modern programming languages like Rust, Go, and Swift provide robust built-in mechanisms for preventing common errors at compile time, APR has proven its potential not only to enhance software reliability and quality but also to streamline maintenance and reduce costs associated with manual debugging. APR tools will continue to serve essential functions in maintaining legacy systems, addressing complex semantic and logical errors that cannot be prevented through type systems alone, and bridging the gap between different programming paradigms and language generations. The integration of APR capabilities with modern language features could lead to hybrid approaches that combine preventive measures at the language level with sophisticated repair mechanisms at runtime. This synergy between APR tools and modern language design principles represents a promising direction for reducing software defects and improving developer productivity.

Future research should focus on developing APR techniques that leverage the safety guarantees provided by modern languages while addressing the remaining challenges in automated program repair, such as complex semantic bugs, cross-language interoperability issues, and runtime optimization problems. As the field continues to advance, research in hybrid approaches, data-efficient learning, enhanced fault localization, and standardized evaluation metrics will be instrumental in fully realizing the potential of APR.

Author contribution T.B. played a pivotal role in defining the research topic, which laid the foundation for the entire study. T.B. provided continuous guidance throughout the research process, offering critical insights and expertise that significantly shaped the project's direction. He also meticulously reviewed and revised the manuscript, enhancing its clarity, accuracy, and overall quality. S.D. was primarily responsible for conducting an extensive literature review, which involved identifying, analyzing, and synthesizing relevant studies to build a solid theoretical basis for the research. S.D. classified various methodologies according to their suitability and effectiveness for the research objectives, ensuring a rigorous analytical approach. S.D. drafted the manuscript, compiled the findings, and articulated the study's contributions and implications.

Funding Open access funding provided by the Scientific and Technological Research Council of Türkiye (TÜBİTAK).

Data availability No datasets were generated or analyzed during the current study.

Declarations

Conflict of interest The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Le XB, Lo D, Le Goues C (2016) History driven program repair. In: 2016 IEEE 23rd international conference on software analysis, evolution, and reengineering (SANER), IEEE, vol 1, pp 213–224
2. Le XB, Chu DH, Lo D, Le Goues C, Visser W (2017) S3: syntax-and semantic-guided repair synthesis via programming by examples. In: Proceedings of the 2017 11th joint meeting on foundations of software engineering, pp 593–604
3. Durieux T, Cornu B, Seinturier L, Monperrus M (2017) Dynamic patch generation for null pointer exceptions using metaprogramming. In: 2017 IEEE 24th international conference on software analysis, evolution and reengineering (SANER), IEEE, pp 349–358
4. Moustafa S, ElNainay MY, El Makky N, Abougabal MS (2018) Software bug prediction using weighted majority voting techniques. *Alex Eng J* 57(4):2763–2774
5. Liu K, Koyuncu A, Kim D, Bissyandé TF (2019) Avatar: fixing semantic bugs with fix patterns of static analysis violations. In: 2019 IEEE 26th international conference on software analysis, evolution and reengineering (SANER), IEEE, pp 1–12
6. Gupta R, Kanade A, Shevade S (2018) Deep reinforcement learning for programming language correction. *arXiv preprint arXiv:1801.10467*
7. Mashhadi E, Hemmati H (2021) Applying codebert for automated program repair of java simple bugs. In: 2021 IEEE/ACM 18th international conference on mining software repositories (MSR), IEEE, pp 505–509
8. Koyuncu A, Liu K, Bissyandé TF, Kim D, Klein J, Monperrus M, Le Traon Y (2020) Fixminer: mining relevant fix patterns for automated program repair. *Empir Softw Eng* 25:1980–2024
9. Gissurarson MP, Applis L, Panichella A, Van Deursen A, Sands D (2022) PropR: property-based automatic program repair. In: Proceedings of the 44th international conference on software engineering, pp 1768–1780
10. Yin P, Neubig G, Allamanis M, Brockschmidt M, Gaunt AL (2018) Learning to represent edits. *arXiv preprint arXiv:1810.13337*
11. Jiang N, Lutellier T, Tan L (2021) Cure: code-aware neural machine translation for automatic program repair. In: 2021 IEEE/ACM 43rd international conference on software engineering (ICSE), IEEE, pp 1161–1173
12. Namavar M, Nashid N, Mesbah A (2022) A controlled experiment of different code representations for learning-based program repair. *Empir Softw Eng* 27(7):190
13. Yasunaga M, Liang P (2021) Break-it-fix-it: unsupervised learning for program repair. In: International conference on machine learning. PMLR, pp 11941–11952
14. Wen M, Chen J, Wu R, Hao D, Cheung SC (2018) Context-aware patch generation for better automated program repair. In: Proceedings of the 40th international conference on software engineering, pp 1–11
15. Xin Q, Reiss SP (2017) Leveraging syntax-related code for automated program repair. In: 2017 32nd IEEE/ACM international conference on automated software engineering (ASE), IEEE, pp 660–670
16. Gazzola L, Micucci D, Mariani L (2019) Automatic software repair: a survey. *IEEE Trans Softw Eng* 45(1):34–67. <https://doi.org/10.1109/TSE.2017.2755013>
17. Zhang Q, Fang C, Ma Y, Sun W, Chen Z (2023) A survey of learning-based automated program repair. *ACM Trans Softw Eng Methodol* 33(2):1–69. <https://doi.org/10.1145/3631974>
18. Huang K, Xu Z, Yang S, Sun H, Li X, Yan Z, Zhang Y (2023) A survey on automated program repair techniques. *arXiv preprint arXiv:2303.18184*

19. Bui ND, Yu Y, Jiang L (2021) Infercode: self-supervised learning of code representations by predicting subtrees. In: 2021 IEEE/ACM 43rd international conference on software engineering (ICSE), IEEE, pp 1186–1197
20. Le Goues C, Nguyen T, Forrest S, Weimer W (2011) Genprog: a generic method for automatic software repair. *IEEE Trans Softw Eng* 38(1):54–72
21. Winter E, Nowack V, Bowes D, Counsell S, Hall T, Haraldsson S, Woodward J (2022) Let's talk with developers, not about developers: A review of automatic program repair research. *IEEE Trans Softw Eng* 49(1):419–436
22. Winter E, Bowes D, Counsell S, Hall T, Haraldsson S, Nowack V, Woodward J (2022) How do developers really feel about bug fixing? directions for automatic program repair. *IEEE Trans Softw Eng* 49:1823–1841
23. Ye H, Martinez M, Durieux T, Monperrus M (2021) A comprehensive study of automatic program repair on the QuixBugs benchmark. *J Syst Softw* 171:110825
24. Zhang S, Li D, Zhao M, Li H, Wong WE (2024) Search-based automated program repair: a survey. In: 2024 IEEE 24th international conference on software quality, reliability, and security companion (QRS-C). IEEE, pp. 447–456
25. Kim D, Nam J, Song J, Kim S (2013) Automatic patch generation learned from human-written patches. In: 2013 35th international conference on software engineering (ICSE), IEEE, pp 802–811
26. Le XB, Chu DH, Lo D, Le Goues C, Visser W (2017) JFIX: semantics-based repair of Java programs via symbolic PathFinder. In: Proceedings of the 26th ACM SIGSOFT international symposium on software testing and analysis, pp 376–379
27. Hua J, Zhang M, Wang K, Khurshid S (2018) Towards practical program repair with on-demand candidate generation. In: Proceedings of the 40th international conference on software engineering, pp 12–23
28. Rolim R, Soares G, Gheyi R, Barik T, D'Antoni L (2018) Learning quick fixes from code repositories. *arXiv preprint arXiv:1803.03806*
29. Liu K, Koyuncu A, Kim K, Kim D, Bissyandé TF (2018) LSRepair: live search of fix ingredients for automated program repair. In: 2018 25th Asia-Pacific software engineering conference (APSEC), IEEE, pp 658–662
30. Liu X, Zhong H (2018) Mining stackoverflow for program repair. In: 2018 IEEE 25th international conference on software analysis, evolution and reengineering (SANER), IEEE, pp 118–129
31. Zhang Q, Fang C, Zhang T, Yu B, Sun W, Chen Z (2023) Gamma: revisiting template-based automated program repair via mask prediction. In: 2023 38th IEEE/ACM international conference on automated software engineering (ASE). IEEE, pp 535–547
32. Saha RK, Lyu Y, Yoshida H, Prasad MR (2017) Elixir: effective object-oriented program repair. In: 2017 32nd IEEE/ACM international conference on automated software engineering (ASE), IEEE, pp 648–659
33. Chen L, Pei Y, Pan M, Zhang T, Wang Q, Furia CA (2022) Program repair with repeated learning. *IEEE Trans Softw Eng* 49(2):831–848
34. Moosavi S, Vahidi-Asl M, Haghighi H, Rezaalipour M (2024) Doctor Code: a machine learning-based approach to program repair. *Sci Iranica* 31(2):83–102
35. Ahmed T, Ledesma NR, Devanbu P (2022) SYNSHINE: improved fixing of syntax errors. *IEEE Trans Softw Eng* 49(4):2169–2181
36. Koyuncu A, Liu K, Bissyandé TF, Kim D, Monperrus M, Klein J, Le Traon Y (2019) iFixR: bug report driven program repair. In: Proceedings of the 2019 27th ACM joint meeting on European software engineering conference and symposium on the foundations of software engineering, pp 314–325
37. Cao H, Liu F, Shi J, Chu Y, Deng M (2023) Random search and code similarity-based automatic program repair. *J Shanghai Jiaotong Univ Sci* 28(6):738–752
38. Valucian M, Vahidi-Asl M, Khalilian A (2022) SituRepair: incorporating machine-learning fault class prediction to inform situational multiple fault automatic program repair. *Int J Crit Infrastruct Protect* 37:100527. <https://doi.org/10.1016/j.ijcip.2022.100527>
39. Yuan Y, Banzhaf W (2018) Arja: automated repair of java programs via multi-objective genetic programming. *IEEE Trans Softw Eng* 46(10):1040–1067
40. Wang S, Lin B, Chen L, Mao X (2024) Divide-and-conquer: Automating code revisions via localization-and-revision. *ACM Trans Softw Eng Methodol*. <https://doi.org/10.1145/3697013>
41. Jiang J, Xiong Y, Zhang H, Gao Q, Chen X (2018) Shaping program repair space with existing patches and similar code. In: Proceedings of the 27th ACM SIGSOFT international symposium on software testing and analysis, pp 298–309
42. Jiang N, Lutellier T, Lou Y, Tan L, Goldwasser D, Zhang X (2023) Knod: domain knowledge distilled tree decoder for automated program repair. *arXiv preprint arXiv:2302.01857*

43. Zhang L, Guo S, Guo Y, Hui Li Y, Chai RC, Li X, Jiang H (2024) Context-based transfer learning for structuring fault localization and program repair automation. *ACM Trans Softw Eng Methodol*. <https://doi.org/10.1145/3705302>
44. Ye H, Martinez M, Monperrus M (2022) Neural program repair with execution-based backpropagation. In: *Proceedings of the 44th international conference on software engineering*, pp 1506–1518
45. Chen Z, Komrusch S, Monperrus M (2022) Neural transfer learning for repairing security vulnerabilities in c code. *IEEE Trans Softw Eng* 49(1):147–165
46. Zhang J, Cambronero JP, Gulwani S, Le V, Piskac R, Soares G, Verbruggen G (2024) Pydex: repairing bugs in introductory python assignments using LLMs. *Proc ACM Programm Lang* 8(OOPSLA1):1100–1124
47. Lajkó M, Csuvik V, Vidács L (2022) Towards JavaScript program repair with generative pre-trained transformer (GPT-2). In: *Proceedings of the Third international workshop on automated program repair*, pp 61–68
48. Santos EA, Campbell JC, Patel D, Hindle A, Amaral JN (2018) Syntax and sensibility: using language models to detect and correct syntax errors. In: *2018 IEEE 25th international conference on software analysis, evolution and reengineering (SANER)*, IEEE, pp 311–322
49. Dinella E, Dai H, Li Z, Naik M, Song L, Wang K (2020) Hoppity: learning graph transformations to detect and fix bugs in programs. In: *International conference on learning representations (ICLR)*
50. Jiang N, Wu Y (2024) RepairCAT: applying large language model to fix bugs in AI-generated programs. In: *Proceedings of the 5th ACM/IEEE international workshop on automated program repair*, pp 58–60
51. Drain D, Clement CB, Serrato G, Sundaresan N (2021) Deepdebug: fixing python bugs using stack traces, backtranslation, and code skeletons. *arXiv preprint arXiv:2105.09352*
52. Ye H, Monperrus M (2024) ITER: iterative neural repair for multi-location patches. In: *Proceedings of the 46th IEEE/ACM international conference on software engineering*, pp 1–13
53. Just R, Jalali D, Ernst MD (2014) Defects4J: a database of existing faults to enable controlled testing studies for Java programs. In: *Proceedings of the 2014 international symposium on software testing and analysis*, pp 437–440
54. Lin D, Koppel J, Chen A, Solar-Lezama A (2017) QuixBugs: a multi-lingual program repair benchmark set based on the Quixey Challenge. In: *Proceedings Companion of the 2017 ACM SIGPLAN international conference on systems, programming, languages, and applications: software for humanity*, pp 55–56
55. Gyimesi P, Vancsics B, Stocco A, Mazinanian D, Beszédes A, Ferenc R, Mesbah A (2019) Bugsjs: a benchmark of javascript bugs. In: *2019 12th IEEE conference on software testing, validation and verification (ICST)*, IEEE, pp 90–101
56. Klabnik S, Nichols C (2019) *The Rust programming language*. No Starch Press, USA
57. Anderson B, Bergstrom L, Goregaokar M, Matthews J, McAllister K, Moffitt J, Sapin S (2016) Engineering the servo web browser engine using Rust. In: *Proceedings of the 38th international conference on software engineering companion*, pp 81–89
58. Donovan AA, Kernighan BW (2015) *The Go programming language*. Addison-Wesley Professional, New York
59. Gallagher K, Gligoric M, Levandowski A (2020) *Swift programming: The big nerd ranch guide*. Pearson Education, UK
60. Lou Y, Benton S, Hao D, Zhang L, Zhang L (2021) How does regression test selection affect program repair? An extensive study on 2 Million Patches. *arXiv preprint arXiv:2105.07311*
61. Shariffdeen R, Mirchev M, Noller Y, Roychoudhury A (2023) Cerberus: a program repair framework. In: *2023 IEEE/ACM 45th international conference on software engineering: companion proceedings (ICSE-Companion)*. IEEE, pp 73–77
62. Sobreira V, Durieux T, Madeiral F, Monperrus M, de Almeida Maia M (2018) Dissection of a bug dataset: Anatomy of 395 patches from defects4j. In: *2018 IEEE 25th international conference on software analysis, evolution and reengineering (SANER)*. IEEE, pp 130–140
63. Bader J, Scott A, Pradel M, Chandra S (2019) Getafix: learning to fix bugs automatically. *Proc ACM Programm Lang* 3(OOPSLA):1–27. <https://doi.org/10.1145/3336058>
64. Vasic M, Kanade A, Maniatis P, Bieber D, Singh R (2019) Neural program repair by jointly learning to localize and repair. *arXiv preprint arXiv:1904.01720*
65. Zhang Q, Fang C, Ma Y, Sun W, Chen Z (2023) A survey of learning-based automated program repair. *ACM Trans Softw Eng Methodol* 33(2):1–6
66. Bhatia S, Singh R (2016) Automated correction for syntax errors in programming assignments using recurrent neural networks. *arXiv preprint arXiv:1603.06129*
67. Chen Z, Hellendoorn VJ, Lamblin P, Maniatis P, Manzagol PA, Tarlow D, Moitra S (2021) PLUR: a unifying, graph-based view of program learning, understanding, and repair. *Adv Neural Inf Process Syst* 34:23089–23101

68. Rahman MM, Watanobe Y, Nakamura K (2021) A bidirectional LSTM language model for code evaluation and repair. *Symmetry* 13(2):247
69. Le Goues C, Holtschulte N, Smith EK, Brun Y, Devanbu P, Forrest S, Weimer W (2015) The ManyBugs and IntroClass benchmarks for automated repair of C programs. *IEEE Trans Softw Eng* 41(12):1236–1256
70. Martinez M, Durieux T, Sommerard R, Xuan J, Monperrus M (2017) Automatic repair of real bugs in java: a large-scale experiment on the defects4j dataset. *Empir Softw Eng* 22:1936–1964
71. Gay G, Just R (2020) Defects4J as a challenge case for the search-based software engineering community. In: *Search-based software engineering: 12th international symposium, SSBSE 2020, Bari, Italy, October 7–8, 2020, Proceedings 12*. Springer International Publishing, pp 255–261
72. Saha RK, Lyu Y, Lam W, Yoshida H, Prasad MR (2018) Bugs. jar: a large-scale, diverse dataset of real-world java bugs. In: *Proceedings of the 15th international conference on mining software repositories*, pp 10–13
73. Majd A, Vahidi-Asl M, Khalilian A, Baraani-Dastjerdi A, Zamani B (2019) Code4Bench: a multidimensional benchmark of Codeforces data for different program analysis techniques. *J Comput Lang* 53:38–52
74. Faruqi M, Pavlick E, Tenney I, Das D (2018) WikiAtomicEdits: a multilingual corpus of Wikipedia edits for modeling language and discourse. *arXiv preprint arXiv:1808.09422*
75. Tufano R, Masiero S, Mastropaolo A, Pascarella L, Poshyvanik D, Bavota G (2022) Using pretrained models to boost code review automation. In: *Proceedings of the 44th international conference on software engineering*, pp 2291–2302. <https://doi.org/10.1145/3510003.3510621>
76. Fan J, Li Y, Wang S, Nguyen TN (2020) A C/C++ code vulnerability dataset with code changes and CVE summaries. In: *Proceedings of the 17th international conference on mining software repositories*, pp 508–512

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.



Sena Dikici is currently pursuing a PhD degree in computer engineering at Bursa Technical University, Turkey, where she also works as a research assistant in the Department of Computer Engineering, Faculty of Engineering and Natural Sciences. She obtained her BSc in computer engineering from Gazi University and completed her MSc at Bursa Technical University. Her research interests include deep learning, data mining, and language models.



Prof. Dr. Turgay Tugay Bilgin received his PhD degree from Marmara University in 2007. He is currently a professor in the Department of Computer Engineering at Bursa Technical University, Faculty of Engineering and Natural Sciences. Previously, he held an academic position at Maltepe University's Faculty of Engineering and Natural Sciences (2014). His research interests include artificial intelligence, big data processing, data mining, and machine learning.